

007 Agile Projectmanagement

Projectmanagement	© Hogeschool PXL
OLOD	42TIN1250 Projectmanagement
Opleiding	Professionele bachelor in de Toegepaste informatica
Departement	PXL-Digital
Lectoren	Lowie Vangaal , Jan Castermans



Wat betekent Agile?



Agile betekent letterlijk *"de behendigheid om snel en makkelijk te bewegen"*.

In projectmanagement is Agile een methodiek die populair is in de computerindustrie, maar ook veelvuldig wordt toegepast in financiële dienstverlening, marketing, industrie, retail en talloze andere sectoren.

Een *"methodiek"* is eenvoudigweg een manier om iets uit te voeren. Het bestaat uit de methodes, technieken en de aanpak om iets daadwerkelijk af te krijgen. In projectmanagement is Agile een van de meest gebruikte methodieken.

Agile begrijp je het best door te kijken hoe het verschilt van de meer traditionele manieren waarop je projectmanagement uitvoert.

Traditioneel projectmanagement is getypeerd door een lineaire aanpak waar achtereenvolgens sommige of alle van de volgende stappen zijn:

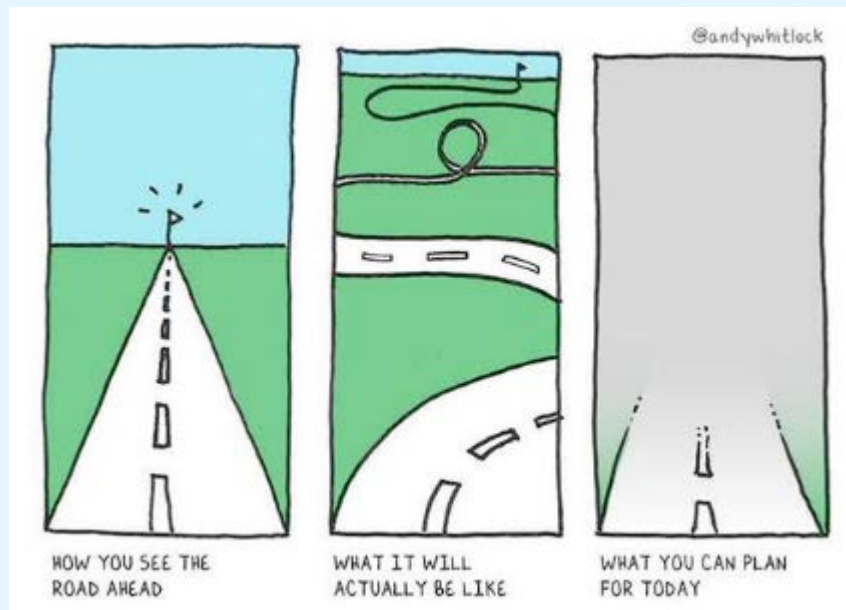
- Initiatie (opstarten van het project)
- Planning
- Uitvoering
- Monitoring
- Afronding

Agile projectmanagement, voor het eerst gebruikt in de late jaren 80, is gekenmerkt door een **niet-lineaire aanpak**. Waar vroeger requirements en oplossingen al in de "Planning"-stap werden afgehandeld, evolueren die nu mee doorheen het proces. Dit gebeurt door samenwerking tussen zelforganiserende, cross-functionele teams.

De uitvoering van projectwerk gebeurt voortdurend doorheen de levensduur van het project. Agile methoden staan voor **adaptieve planning**, **evolutionaire ontwikkeling** en **continue verbetering**. In andere woorden: we zien aanpasbare planningen, we ontwikkelen op een geleidelijke manier en we omarmen continuous improvement.

Agile moedigt een snelle en flexibele reactie op verandering aan.

Het spitst zich toe op teamwerk en op het één-voor-één afwerken van producteigenschappen. Men gaat één eigenschap volledig afwerken voordat naar een volgende eigenschap wordt overgegaan.



Afbeelding: De enige zekerheid in een project is dat alles onzeker is

Samengevat

Agile is een manier van projectmanagement die zich toespitst op het verdelen van taken in korte werkfasen, met veelvuldige reviews (controles) van het project en mogelijke aanpassingen tijdens het uitvoeren. Een Agile aanpak is flexibel en minder strak dan sommige andere manieren van projectmanagement.

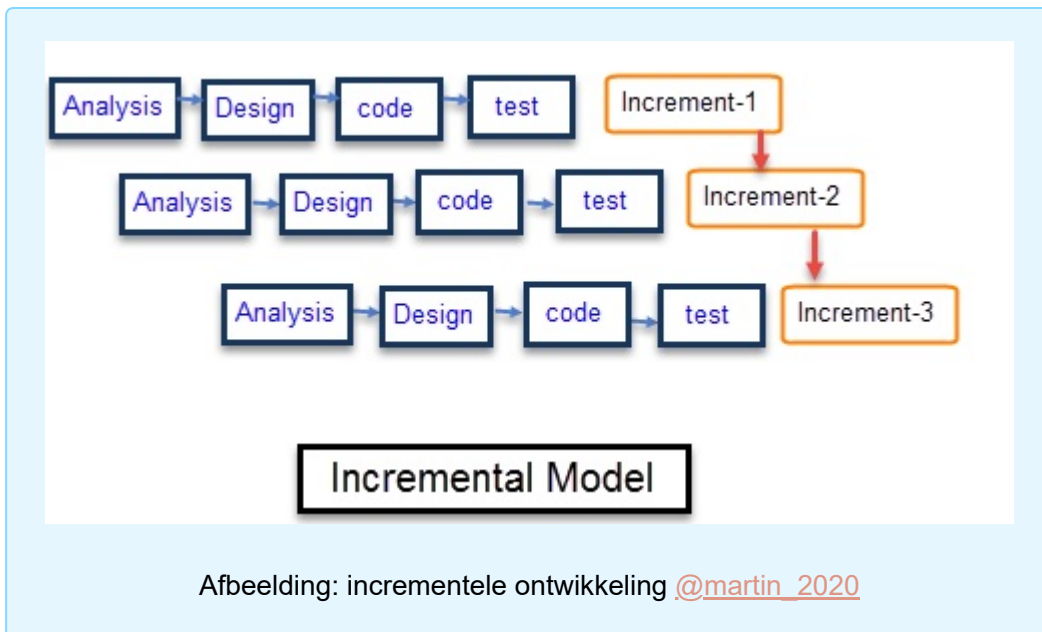
[@stanley.etal_2020](#)

Incrementele ontwikkeling

Een van de eigenschappen van Agile is dat het **incrementeel** is.

DEFINITIE: Increment

Een Increment is een kleine stap. **Incrementeel** slaat op het voorwaarts bewegen in fasen.



Incrementele ontwikkeling is een praktijk waar een product stap voor stap wordt gededignd, gedeployd en getest, totdat het project is afgerond. Telkens wordt er een klein beetje meer toegevoegd. Elke stap bouwt verder op de vorige door extra functionaliteiten toe te voegen.

Later in de cursus zie je [User Stories en Epics](#), die het perfecte voorbeeld van een Increment zijn.

Incrementeel: Wat wordt er gebouwd? → Toevoegen van functies in stappen.

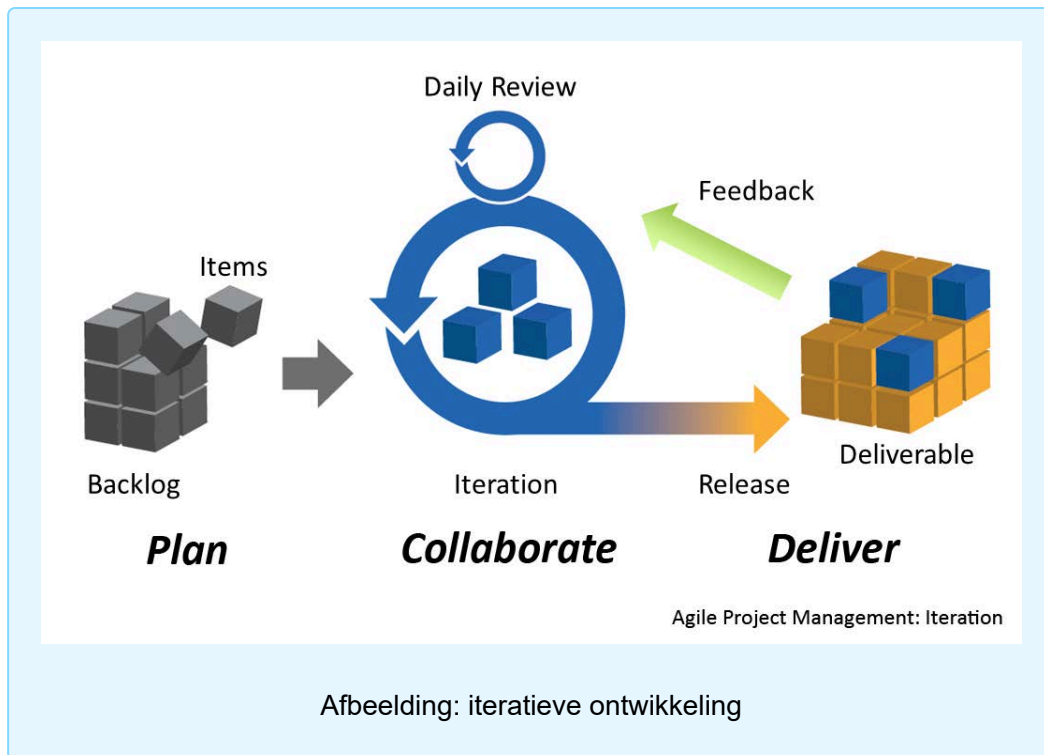
Iteratieve ontwikkeling

Een andere eigenschap van Agile is dat het iteratief is.

DEFINITIE: Iteratief

Iteratief betekent dat we het herhaald gaan uitvoeren.

Iteratieve ontwikkeling slaat op de ontwikkeling van producten door herhaalde cyclussen. Elk van deze cyclussen voltooit een of ander minimum bruikbaar product (*vert. Engels: minimum viable product*).



[1]

Bijvoorbeeld:

- 1e iteratie: Maak een simpele homepage.
- 2e iteratie: Voeg de mogelijkheid voor een gebruiker toe om een account te maken en in te loggen.
- 3e iteratie: Voeg de mogelijkheid toe om je wachtwoord te veranderen.
- 4e iteratie: Voeg een pagina toe die accountinformatie toont.
- enz.

Op het eind van elke cyclus is er een afgewerkt product beschikbaar en kan men het product gebruiken.

Later in de cursus zie je [Sprints](#), die het perfecte voorbeeld van een Iteratie zijn.

Iteratief: Hoe wordt er gebouwd? → Verfijnen van het product in cycli.

Scrum



[2]

Scrum is een van de mogelijke uitvoeringen van Agile.

De term 'Scrum' komt van een uitdrukking uit de rugbysport waar de teamleden elkaars armen in elkaar laten grijpen en zo voorwaarts richting de tegenstander drukken.

Het gros van de huidige development projecten gebruikt Scrum. Ondanks de verzameling termen en technieken gebruikt in Scrum, draait het allemaal rond teamwerk, het samenspel tussen de leden van het team.

Als het beste team een taak in één week kon voltooien, hoe lang denk je dan dat het slechtste team erover deed? Misschien gok je op dezelfde verhouding die bij het Yale-onderzoek werd gevonden, tien op één (oftewel, het trage team deed meer dan twee maanden over iets wat het snelle team in een week af kreeg). Maar het echte antwoord is dat de verschillen bij teamprestaties veel groter zijn dan bij individuele prestaties. Het trage team deed in werkelijkheid niet tien weken over de taak die het beste team in een week afrondde; het was véél erger: het duurde tweeduizend weken!

Scrum
Jeff Sutherland

Je Sutherland
(grondlegger)

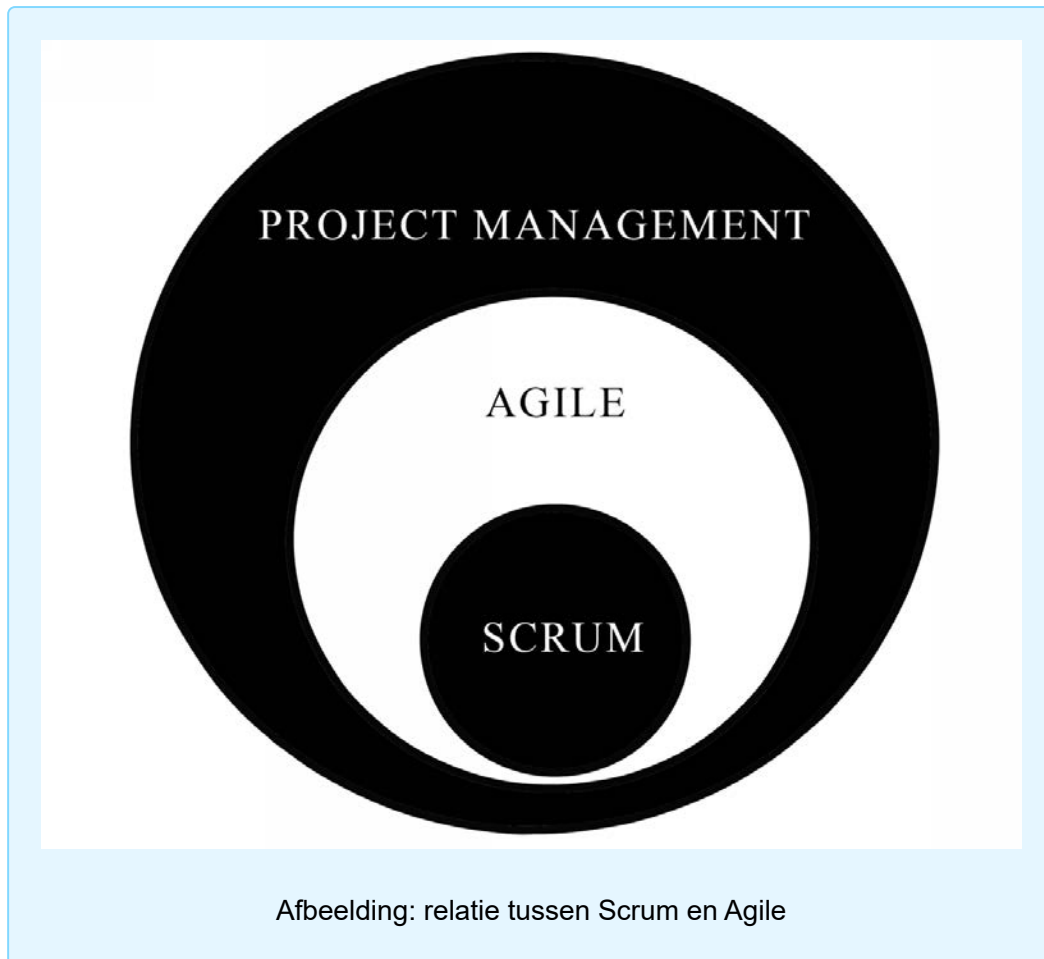
TWEEMAAL ZOVEEL DOEN IN

SCRUM

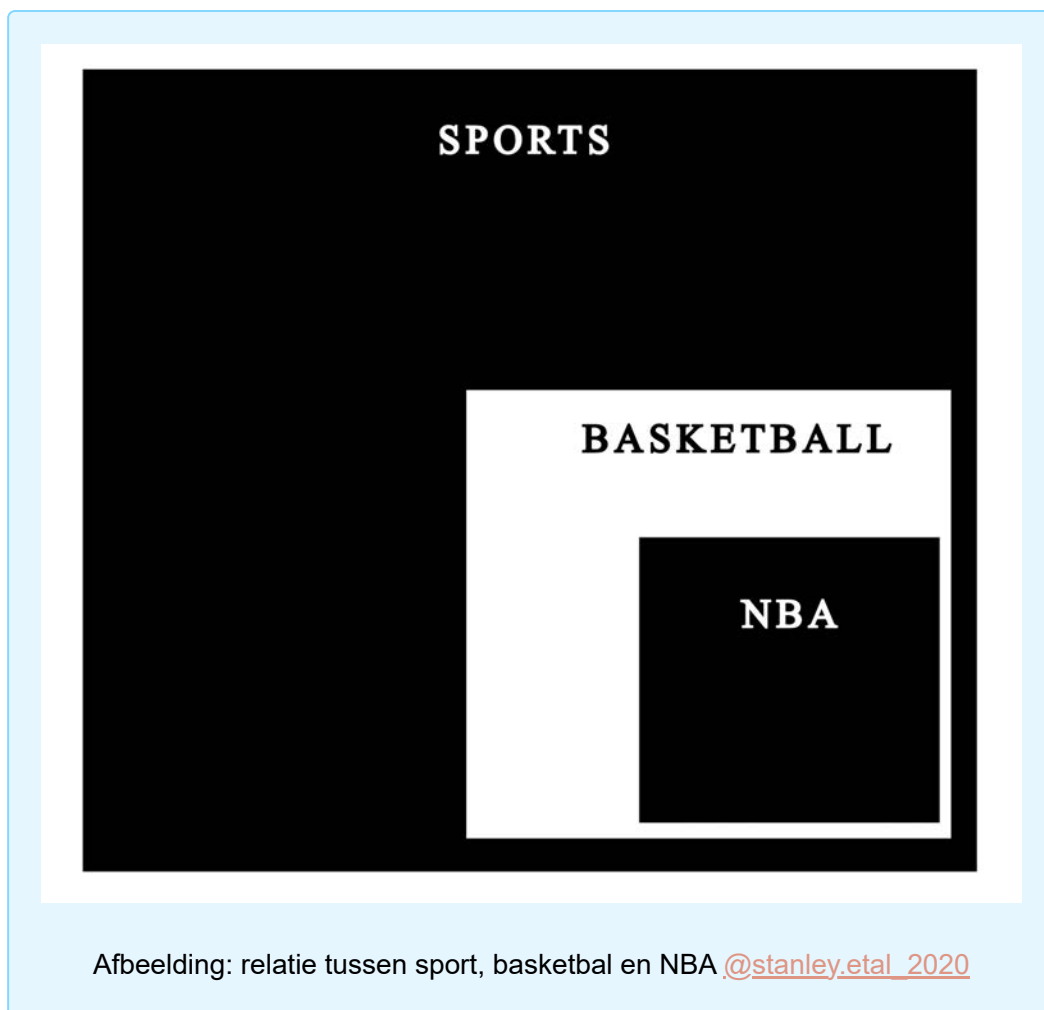
Hoewel Scrum oorspronkelijk is bedacht voor softwareontwikkeling, wordt het toegepast in vele andere sectoren, zoals gezondheidszorg, marketing, rechtshandhaving, productontwikkeling en de publieke sector op alle niveaus.

Waar we Agile beschouwen als een methodologie, spreken we bij Scrum over een raamwerk (*vert. Engels: framework*). De reden dat we dat vermelden is zodat je weet dat er technisch een verschil is tussen beide termen.

Een goede manier om de relatie tussen Scrum en Agile te bekijken is de volgende afbeelding.

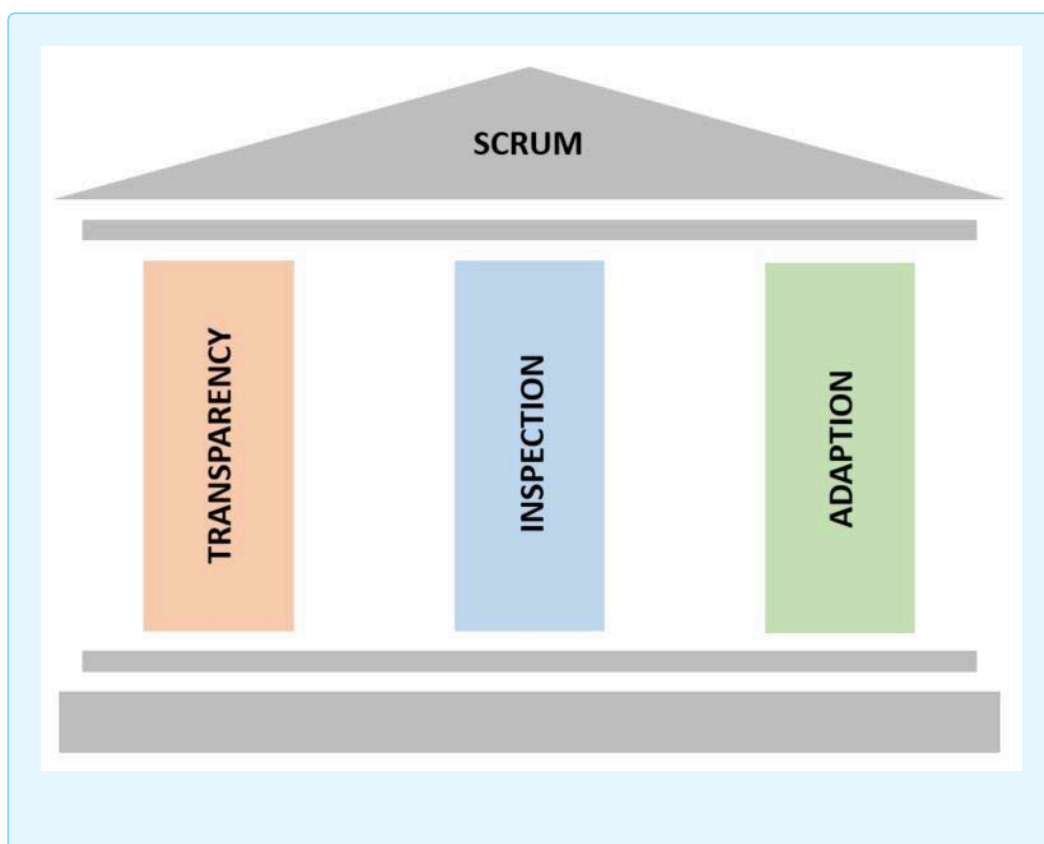


Of, om het met een analogie te bekijken.



In dit voorbeeld bekijk je "sports" als projectmanagement, "basketbal" als Agile en "NBA" (de populairste uitvoering van basketbal) als Scrum.

Empirische procescontrole



Afbeelding: Scrum en empirisme [@zentaoteam_2022a](#)

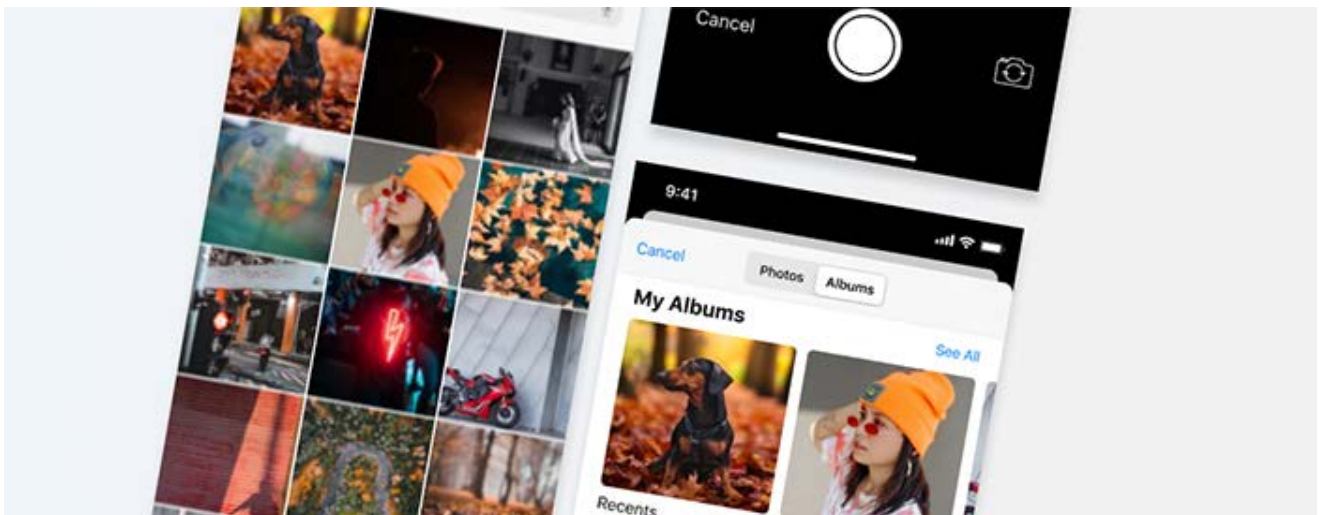
Empirische procescontrole: Leren door te doen en te observeren.

Scrum is gebaseerd op een simpel idee: we leren het beste door dingen te doen en te kijken wat er gebeurt. Dit noemen we empirisme. Het betekent dat we niet alleen vertrouwen op wat we denken te weten, maar echt kijken naar wat er in de praktijk gebeurt.

In Scrum gebruiken we de drie empirische principes om dit in de praktijk te brengen:

- **Transparantie:** Iedereen ziet wat er gebeurt
 - We zijn eerlijk over hoe het werk vordert.
 - We verbergen geen problemen of uitdagingen.
 - Iedereen in het team weet wat er gaande is.
- **Inspectie:** We kijken regelmatig hoe het gaat
 - We checken vaak of we op de goede weg zijn.
 - We laten anderen (zoals de klant) zien wat we gemaakt hebben.
 - We vragen om feedback om te leren en te verbeteren.
- **Aanpassing:** We passen ons aan als dat nodig is
 - Als iets niet werkt, veranderen we onze aanpak.
 - We zijn flexibel en bereid om nieuwe ideeën te proberen.
 - We verbeteren continu onze manier van werken.

Door deze principes toe te passen, leert het team voortdurend. We volgen niet blindelings een plan, maar kijken steeds of wat we doen echt werkt. Als iets beter kan, passen we het aan. Zo worden we steeds beter in wat we doen en leveren we betere resultaten voor de klant.



Stel je voor dat je voor de PXL een nieuwe app ontwikkelt voor het delen van foto's. In plaats van maanden te besteden aan het plannen en bouwen van de perfecte app, ga je als volgt te werk:

1. Je maakt een eenvoudige versie van de app die alleen foto's kan uploaden (**Transparantie**: je hebt iets concreets om te laten zien).
2. Je laat een kleine groep gebruikers de app testen (**Inspectie**: je observeert hoe de app wordt gebruikt).
3. De gebruikers melden dat ze graag filters aan hun foto's willen toevoegen voordat ze deze uploaden (**Aanpassing**: je leert van de feedback).
4. Je voegt een eenvoudige filterfunctie toe aan de app (Je past de app aan op basis van wat je hebt geleerd).
5. Je herhaalt stappen 2-4: laat gebruikers testen, verzamel feedback, en pas de app aan (Iteratie: je blijft dit proces herhalen).

Empirisch denken kan je op die manier begrijpen:

- Je baseert je beslissingen op echte ervaringen en observaties (empirisch bewijs), niet op theorieën of aannames.
- Je bent transparant over wat de app wel en niet kan.
- Je inspecteert regelmatig hoe de app wordt gebruikt.
- Je past de app aan op basis van wat je leert.

Dit is de kern van hoe Scrum werkt: we doen, we kijken, we leren, en we verbeteren. Het is een cyclus die steeds doorgaat, waardoor we flexibel blijven en snel kunnen reageren op veranderingen.

Het kan geen kwaad deze uit te printen, ze op je nachtkastje te leggen, ze af en toe te lezen en een traantje weg te pinken. Beschouw ze als mogelijk de waardevolste lessen die je dankzij Scrum kan leren.

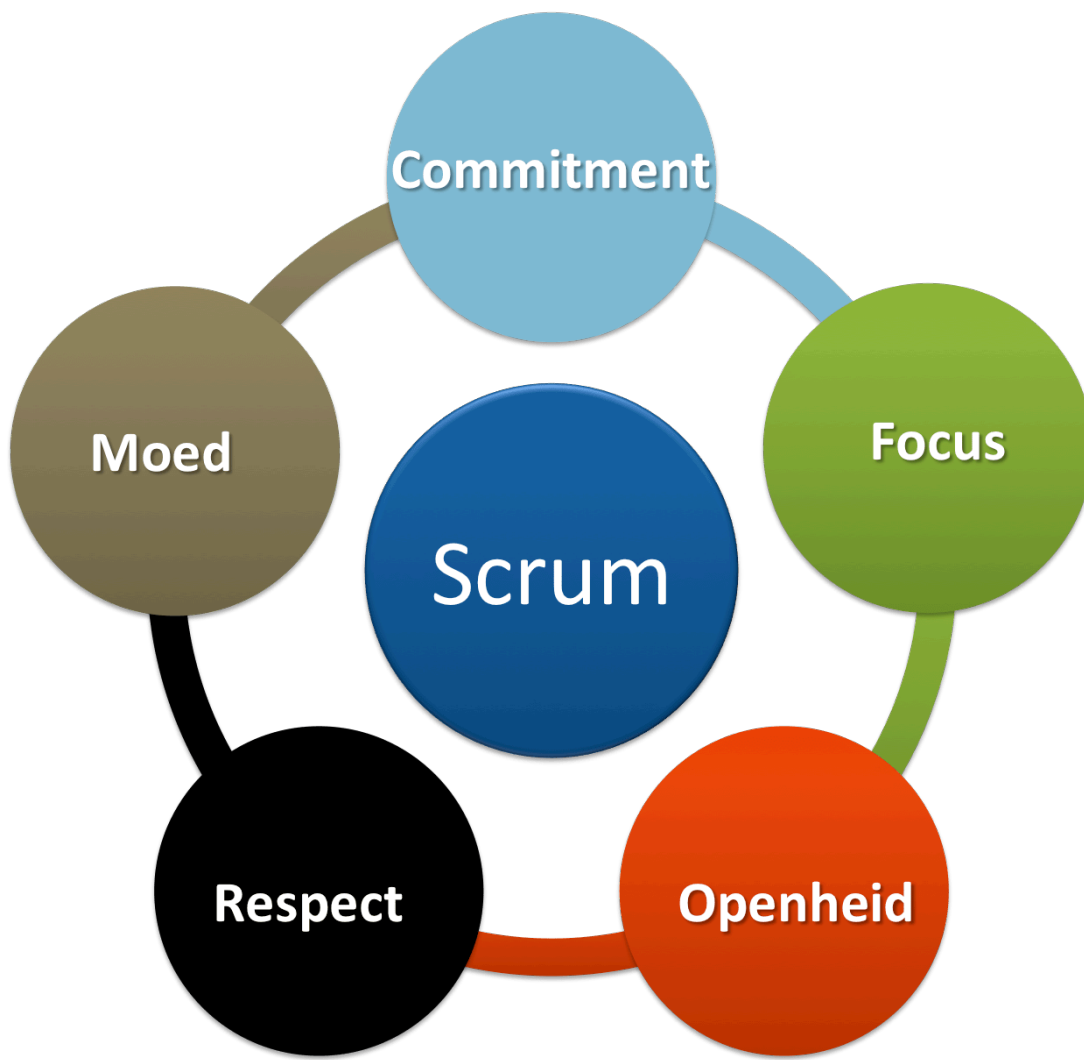
Tip

Hierna, in de [Scrum Waarden](#), gaan we dieper in op de manier waarop de empirische gedachte in Scrum is uitgewerkt.

Scrum Waarden

DEFINITIE: Scrum Waarden

Scrum Waarden: vijf kernwaarden waar Scrum op steunt: commitment, focus, openheid (*Engels: 'openness'*), respect en moed (*vert. Engels: 'courage'*).



Scrum Waarden: De basis voor goed teamwork

Scrum is gebaseerd op vijf kernwaarden die essentieel zijn voor een goede samenwerking:

Commitment (Toewijding)

- We zetten ons volledig in voor ons werk en onze doelen.
- We committeren ons aan het team en de Sprintdoelen, niet aan specifieke oplossingen.
- We streven naar kwaliteit en blijven leren.
- We zijn toegewijd aan continue verbetering.

Focus

- We concentreren ons op wat nu het belangrijkste is voor de Sprint en productdoelen.
- We werken aan een beperkt aantal taken tegelijk om effectiever te zijn.
- We minimaliseren afleidingen en houden ons aan de Scrum-processen.

Openheid

- We zijn transparant over ons werk, voortgang en uitdagingen.
- We delen informatie actief, ook als het niet zo goed gaat.

- We staan open voor nieuwe ideeën, feedback en veranderingen.
- We creëren een omgeving waar iedereen zich vrij voelt om te spreken.

Respect

- We waarderen elkaars meningen, vaardigheden en achtergronden.
- We respecteren elkaar als bekwame, autonome mensen.
- We respecteren de Scrum-regels, -rollen en de belangen van stakeholders.
- We tonen respect door effectief samen te werken en elkaar te ondersteunen.

Moed

- We durven moeilijke beslissingen te nemen en problemen aan te kaarten.
- We zijn eerlijk over vooruitgang en schattingen.
- We experimenteren met nieuwe aanpakken om te verbeteren.
- We erkennen fouten en leren ervan, in plaats van ze te verbergen.

Deze waarden versterken elkaar en zijn cruciaal voor het succes van Scrum. Ze helpen teams om:

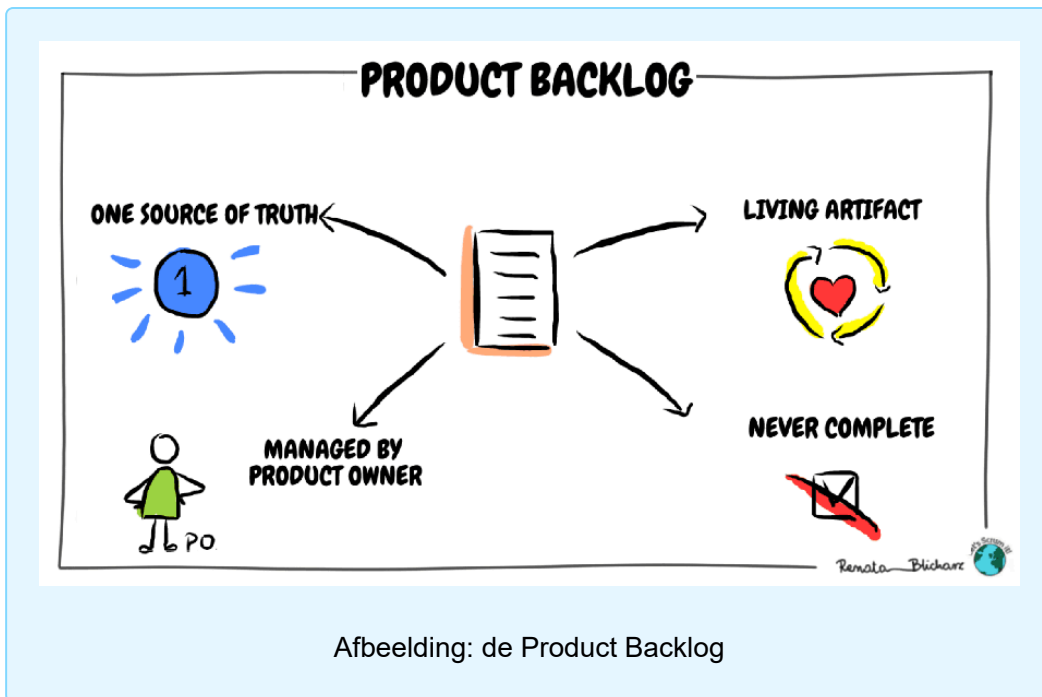
- Vertrouwen op te bouwen en betere samenwerking te bevorderen.
- Transparanter en effectiever te communiceren.
- Flexibeler te reageren op veranderingen en uitdagingen.
- Een cultuur van continue verbetering te creëren.
- Hoogwaardige producten te leveren die echte waarde bieden aan klanten.

Door deze waarden telkens toe te passen, maken Scrum-teams een werkomgeving waar iedereen kan uitblinken en waar je samen de beste resultaten kan bereiken.

The Five Scrum Values (Video)

Product Backlog

Je hebt zeker al het woord 'backlog' horen vallen wanneer het gaat over achterstallig werk, of over werk dat over tijd is. Bijvoorbeeld als je al een maand lang je mails niet hebt beantwoord, zou je een backlog van e-mails hebben.



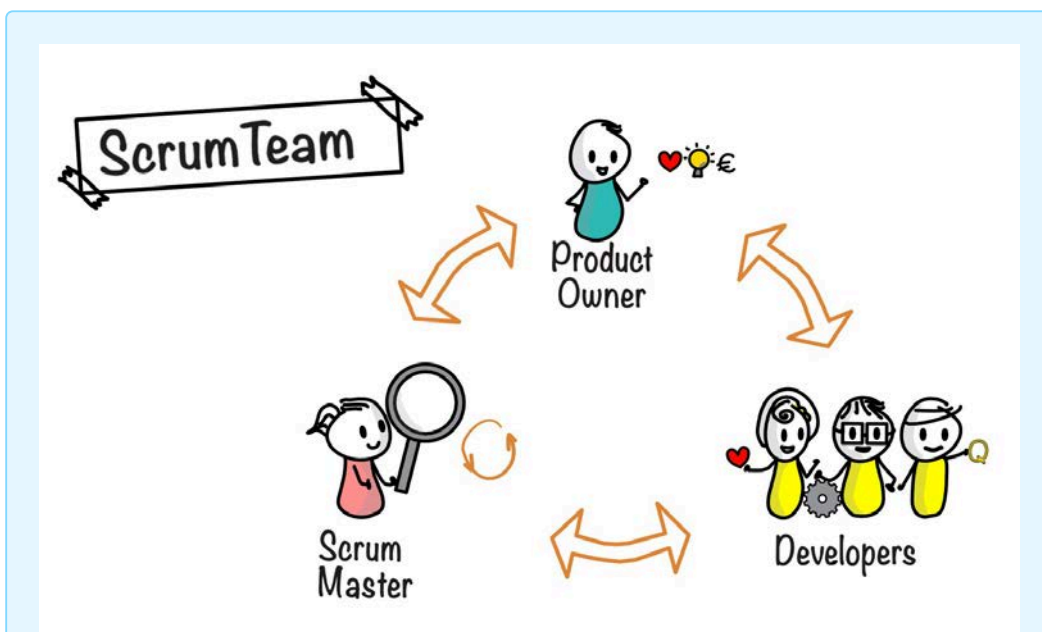
In Agile definieer je een backlog als volgt:

DEFINITIE: Product Backlog

Product Backlog: het openstaande werk. Elke taak die nog verondersteld wordt te gebeuren, beschouwen we als backlog. Het slaat op het aankomend werk dat nog niet aan een persoon toegewezen is of waarvan de afwerkingsdatum nog niet is vastgelegd.

Als je, bijvoorbeeld, software aan het schrijven bent waarmee je de leden, locaties en rooster van een jeugdbeweging wil vastleggen, zou je wat je moet doen om deze functionaliteit werkelijk te bouwen, opsplitsen in verscheidene individuele taken. Deze lijst is je Product Backlog. Telkens je een taak afgewerkt hebt, zal die verdwijnen van de backlog.

De drie Rollen in Scrum



Afbeelding: de drie rollen in Scrum

De drie hoofdrollen in Scrum zijn:

1. Product Owner
2. Scrum Master
3. Developers

Product Owner

De Product Owner vormt de brug tussen de klant en het ontwikkelteam. De Product Owner is er om ervoor te zorgen dat het team precies begrijpt wat de klant wil en waarom.

Een van de belangrijkste taken van de Product Owner is het beheren van de Product Backlog.

De Product Owner bepaalt welke taken op deze lijst komen en in welke volgorde ze moeten worden aangepakt. Hij of zij zorgt ervoor dat de belangrijkste taken bovenaan staan, zodat het team altijd weet waar ze als eerste aan moeten werken.

De rol van Product Owner draait om communicatie. Product Owners praten veel met de klant om te begrijpen wat er precies nodig is. Vervolgens vertalen ze deze wensen naar duidelijke, werkbare taken voor het team. Als het team vragen heeft over een taak, staat de Product Owner klaar om uitleg te geven.

De Product Owner is ook verantwoordelijk voor het beoordelen van het werk dat het team oplevert. Na elke Sprint kijkt de Product Owner of alles voldoet aan de verwachtingen van de klant. Ze geven feedback en beslissen of iets klaar is om aan de klant te laten zien.

Het is belangrijk om te begrijpen dat de Product Owner geen baas is die het team vertelt wat ze moeten doen. In plaats daarvan werken ze nauw samen met het team. Ze helpen het team om prioriteiten te stellen en zorgen ervoor dat iedereen begrijpt waarom bepaalde taken belangrijk zijn.

Door de rol van de Product Owner wordt het hele proces efficiënter. Het team weet precies wat ze moeten maken en waarom. Er wordt minder tijd verspild aan taken die niet belangrijk zijn voor de klant. En omdat er voortdurend feedback wordt gegeven, wordt het eindproduct steeds beter afgestemd op wat de klant echt nodig heeft.



DEFINITIE: Product Owner

Product Owner: De Product Owner in Scrum is verantwoordelijk voor het bepalen van wat het team moet bouwen, de taken te prioriteren volgens belangrijkheid en waarde, en te zorgen dat het team altijd duidelijke richtlijnen heeft om het juiste product te ontwikkelen.

Scrum Master



[@qframe_2021](#)

Rol en doel

In het hart van elk Scrum-team vinden we de Scrum Master. Deze persoon is veel meer dan alleen een teamleider; ze helpen het team Scrum goed te gebruiken. Hun doel is om ervoor te zorgen dat Scrum correct wordt toegepast en dat het team zo efficiënt mogelijk kan werken.

Dagelijkse betrokkenheid

De Scrum Master is diep betrokken bij de dagelijkse activiteiten van het team. Scrum Masters helpen bij het coördineren van groepsactiviteiten en zorgen ervoor dat alles soepel verloopt. Een belangrijk deel van hun werk is het identificeren en verwijderen van obstakels die het team tegenhouden. Dit kunnen technische problemen zijn, maar ook communicatieproblemen of andere zaken die de voortgang belemmeren.

Faciliteren van Scrum-events

Een van de kernverantwoordelijkheden van de Scrum Master is het faciliteren van de Scrum-events. Ze zorgen ervoor dat de [Daily Standups](#), [Sprint Planning](#)-sessies, [Sprint Reviews](#) en [Sprint Retrospectives](#) (zie later) effectief verlopen en binnen de tijdslimieten blijven. Tijdens deze bijeenkomsten moedigt de Scrum Master open communicatie aan en helpt het team om zich te concentreren op de belangrijkste zaken.

Coaching en training

De Scrum Master is ook als een coach voor het team. Ze helpen teamleden om Scrum beter te begrijpen en toe te passen. Dit kan betekenen dat ze trainingen geven, vragen beantwoorden, of gewoon het goede voorbeeld geven hoe Scrum zou moeten werken. Ze moedigen het team aan om zelforganiserend te zijn en helpen bij het ontwikkelen van vaardigheden die nodig zijn voor effectief teamwerk.

Relatiebeheer

Buiten het team speelt de Scrum Master een belangrijke rol in het onderhouden van relaties. Ze werken nauw samen met de Product Owner om ervoor te zorgen dat de [Product Backlog](#) effectief wordt beheerd. Ze zijn een buffer tussen het team en externe verstoringen, zodat het team zich kan concentreren op hun werk.

Ondersteunende rol

De Scrum Master is geen traditionele "manager" die taken toewijst of controleert. In plaats daarvan zijn ze er om het team te ondersteunen en in staat te stellen hun beste werk te leveren. Ze creëren een omgeving waarin het team kan floreren, problemen kan oplossen en continu kan verbeteren.

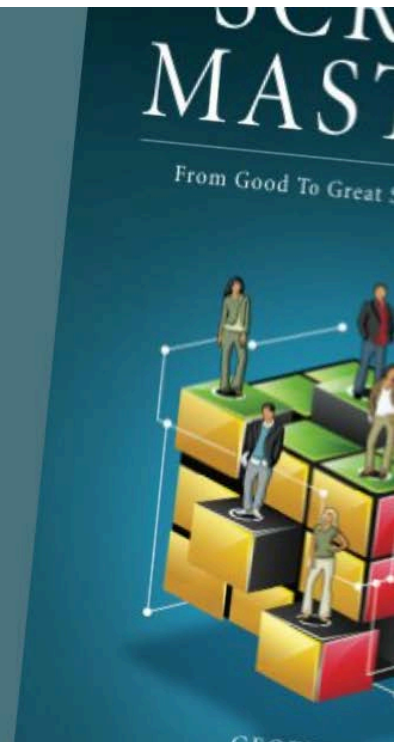
DEFINITIE: Scrum Master

Scrum Master: de hoeder van het Scrum-proces, een facilitator voor het team, en een brug naar de rest van de organisatie. Hun werk zorgt ervoor dat Scrum effectief wordt toegepast, wat leidt tot betere samenwerking, hogere productiviteit en uiteindelijk betere resultaten voor het project.

De term "*master*" kenmerkt in deze context niet iemand die anderen voor zich laat werken. Het is eerder te verstaan als "*een specialist van een bepaald onderwerp*". De Scrum Master is iemand die het beste (of zeker een sterk) begrip van Scrum heeft, vergeleken met de rest van de mensen betrokken bij het project. Men beschouwt hen als de '*masters van het Scrum-raamwerk*'.

A ScrumMaster is part facilitator, part coach and part coordinator. They are also part parent, part orchestra conductor and part sheepdog. And much, much more. The ScrumMaster should do whatever is needed to help the team become high performing and for the organisation to deliver excellent products quickly.

Scrum Mastery
Geoff Watts



Obstakel



Een **obstakel** (*synoniem: belemmering, hindernis of impediment*) is iets wat vooruitgang blokkeert of vertraagt.

In een letterlijke betekenis kun je een omgevallen boom die een straat blokkeert een "obstakel" noemen.

In Scrum is een obstakel eender wat dat het team tegenhoudt hun volledige potentieel te gebruiken.

DEFINITIE: obstakel

Obstakel: elke hindernis die het ontwikkelwerk blokkeert of vertraagt, maar niet kan weggenomen worden via zelfsturing. De verwijdering ervan is onderdeel van de

aansprakelijkheid van de Scrum Master. [@verheyen_2022](#).

Dit omvat problemen die een project tegenhouden het op tijd en tegen het juiste budget te laten beëindigen. Het is de taak van de [Scrum Master](#) deze obstakels te behandelen of te verwijderen.

Voorbeelden van obstakels: zieke teamleden, ontbrekende resources, afwezigheid van management-support, een te koude werkruimte, conflicten tussen teamleden, problemen met leveranciers, gebrekkige technische kennis.

Belangrijk

Bij vraag 3 van de [Daily Scrums](#) ("Welke mogelijke obstakels belemmeren mijn vooruitgang?") is het de morele verplichting elk nieuw obstakel te signaleren. Dit laat toe blokkades zo snel mogelijk op te lossen.

De Developers



De Developers (*synoniem: Ontwikkelteam of Development Team*) bestaat uit professionals die werken aan de taken die horen bij de ontwikkeling van het product. Deze taken bepaalt men voor elke Sprint in overleg met de Product Owner, die de taken selecteert uit de [Product Backlog](#). Aan het eind van de Sprint is het de bedoeling dat de Developers alle taken die geselecteerd waren hebben uitgevoerd.

Kenmerken van de Developers

De Developers hebben enkele kenmerken:

- Het team is zelforganiserend. Niemand vertelt het team hoe de geselecteerde [Product Backlog items](#) uit te voeren, zelfs de [Scrum Master](#) niet.
- Er is een grote mate van persoonlijk leiderschap.

- Het team is cross-functioneel. De leden van het team bezitten samen minimaal 80% van de vaardigheden om de taken uit te voeren.
- Het team is één. Het is niet toegestaan subteams te maken; het team blijft altijd bij elkaar.
- Het team is samen verantwoordelijk. Als iemand binnen het team omwille van benodigde vaardigheden bepaalde taken uitvoert, is het team alsnog gezamenlijk verantwoordelijk voor het eindresultaat.

Aantal teamleden

Hoeveel leden een team bevat, hangt natuurlijk af van de organisatie en het project. Over het algemeen geldt dat een team moet bestaan uit 3 tot 10 leden. Minder dan drie mensen in het team verkleint de interactie, wat leidt tot lagere productiviteit. Daarnaast mist men mogelijk bepaalde vaardigheden in het team, waardoor er tegen problemen wordt aangelopen. Meer dan 10 mensen in het team vereist strakkere coördinatie en zorgt voor hogere complexiteit.

De **Product Owner** en de **Scrum Master** horen niet tot de Developers, behalve als ze taken uit de **Product Backlog** uitvoeren.

[@scrumguide_2022](#)

Terminologie: Scrum Master, Product Owner en Projectmanager

De termen "Scrum Master", "Product Owner" en "projectmanager" worden vaak door elkaar gehaald. Laten we het verschil verduidelijken:

Scrum Master

- Lid van het ontwikkelteam
- Zorgt dat het team Scrum goed gebruikt
- Helpt het team samen te werken

Product Owner

- Vertegenwoordigt de klant
- Beslist wat er gemaakt moet worden
- Praat rechtstreeks met de klant

Projectmanager

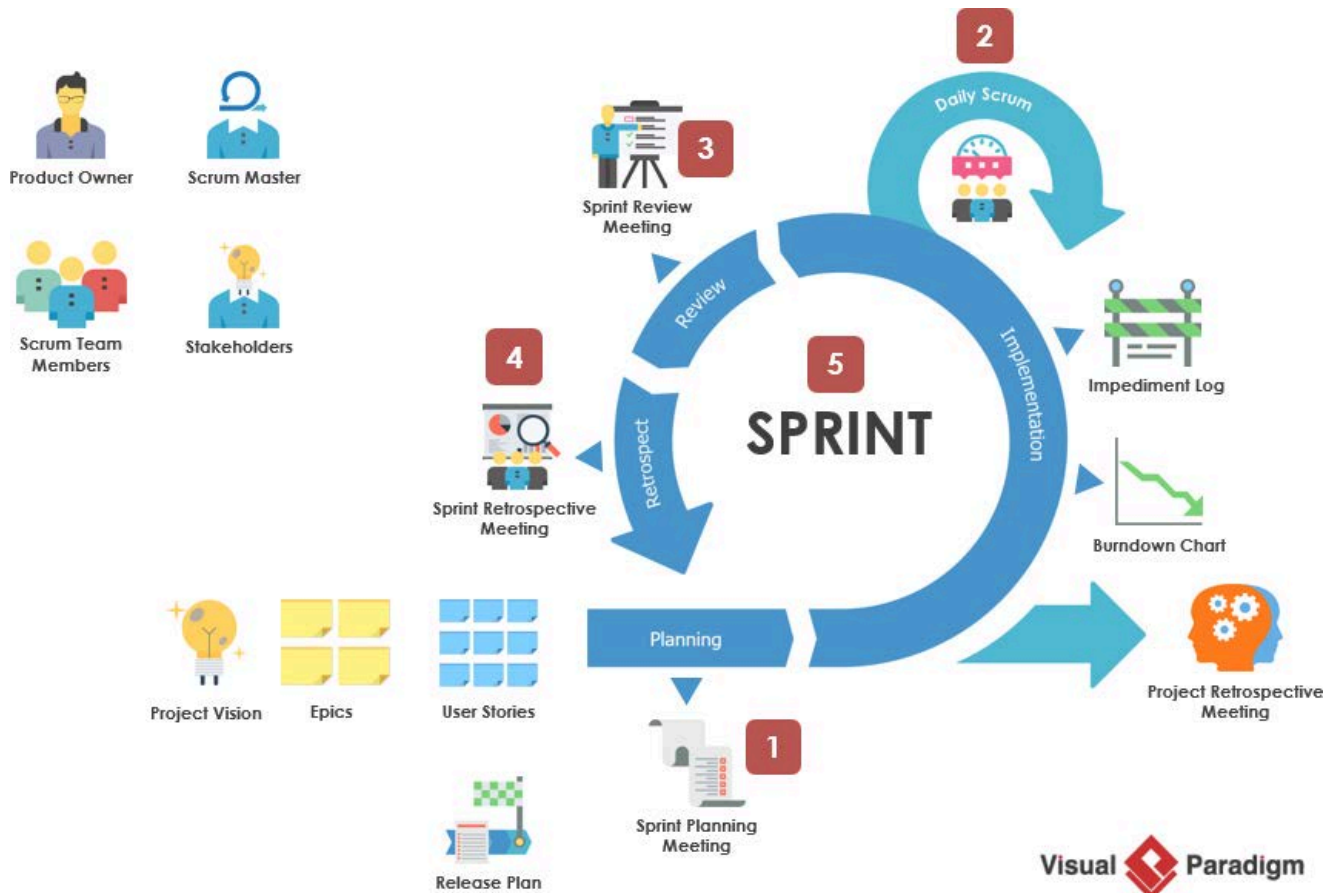
- Deze rol bestaat niet in Scrum
- In Scrum verdelen de Product Owner en Scrum Master deze taken

Let op:

- "Scrum Master" is een term die alleen in Scrum gebruikt wordt

- "Product Owner" komt uit Agile, maar wordt ook in Scrum gebruikt
- "Projectmanager" is een algemene term die in veel verschillende beroepen voorkomt

Overzicht van de 5 Scrum Events



[@visualparadigm_2022](#)

Hoe hangt Scrum nu aan elkaar? En welke zijn de vijf Scrum Events?

Samengevat

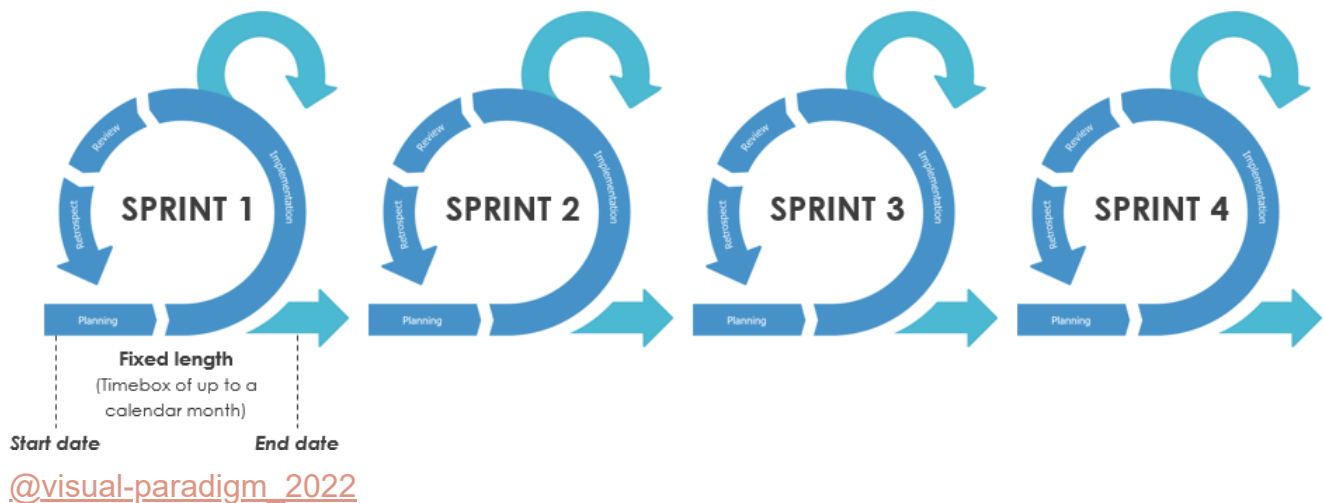
De vijf Scrum Events zijn:

1. De Sprint Planning-meeting
2. De Daily Scrums (of "Daily Standups")
3. De Sprint Review-meeting
4. De Sprint Retrospective-meeting
5. De Sprint

De **Sprint** begint met de **Sprint Planning** waarin we plannen wat we gaan maken, we hebben de dagelijkse **Daily Scrums** waarin we mogelijke obstakels zo vlug mogelijk oplossen. Op het eind van een Sprint wordt getoond wat er gemaakt werd tijdens de **Sprint Review** en ten slotte gaat het team tijdens de **Sprint Retrospective** terugblikken naar de voorbije Sprint om te bepalen hoe hun werkwijzen nog te verbeteren.

Merk op dat de Product Backlog Refinement – al is het een gebruikelijk event, en zeker geen onbelangrijk – niet tussen de vijf officiële Scrum Events staat.

Sprints



Sprints zijn de hartslag van Agile, waar ideeën worden omgezet in waarde.

Een **Sprint** is een vaste periode (meestal 1, 2 of 4 weken) waarin een specifieke hoeveelheid werk wordt uitgevoerd. Zodra een Sprint eindigt, begint er meteen een nieuwe. We itereren!

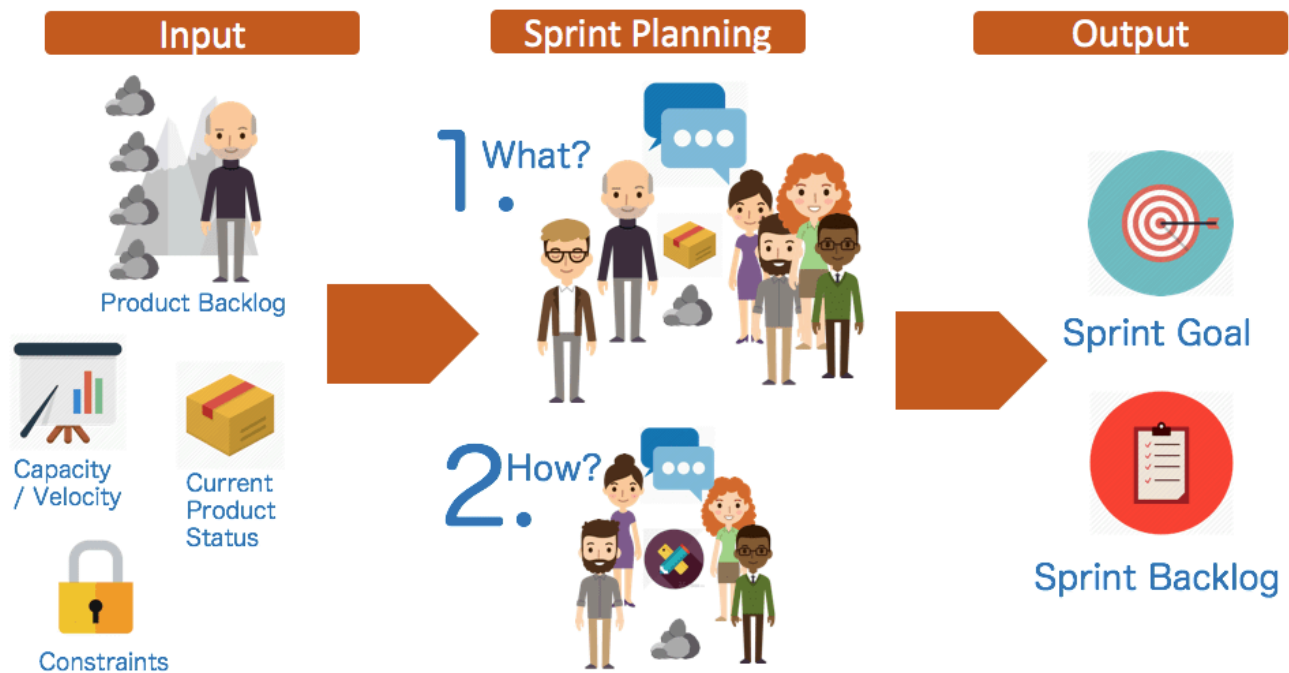
Elke Sprint start met vergaderingen waarin taken verdeeld worden, en eindigt met een review, waarin het uitgevoerde werk wordt bekeken en besproken.

In Agile worden projecten verdeeld in Sprints. Elke dag tijdens een Sprint zijn er Daily Standups om de voortgang te bespreken.

Het Sprint Doel is de centrale focus en het einddoel van een Sprint. Dit wordt ook wel het 'thema' van de Sprint genoemd.

Sprint Planning

Bij de start van elke Sprint, is er een planningssessie binnen het team: de **Sprint Planning**.



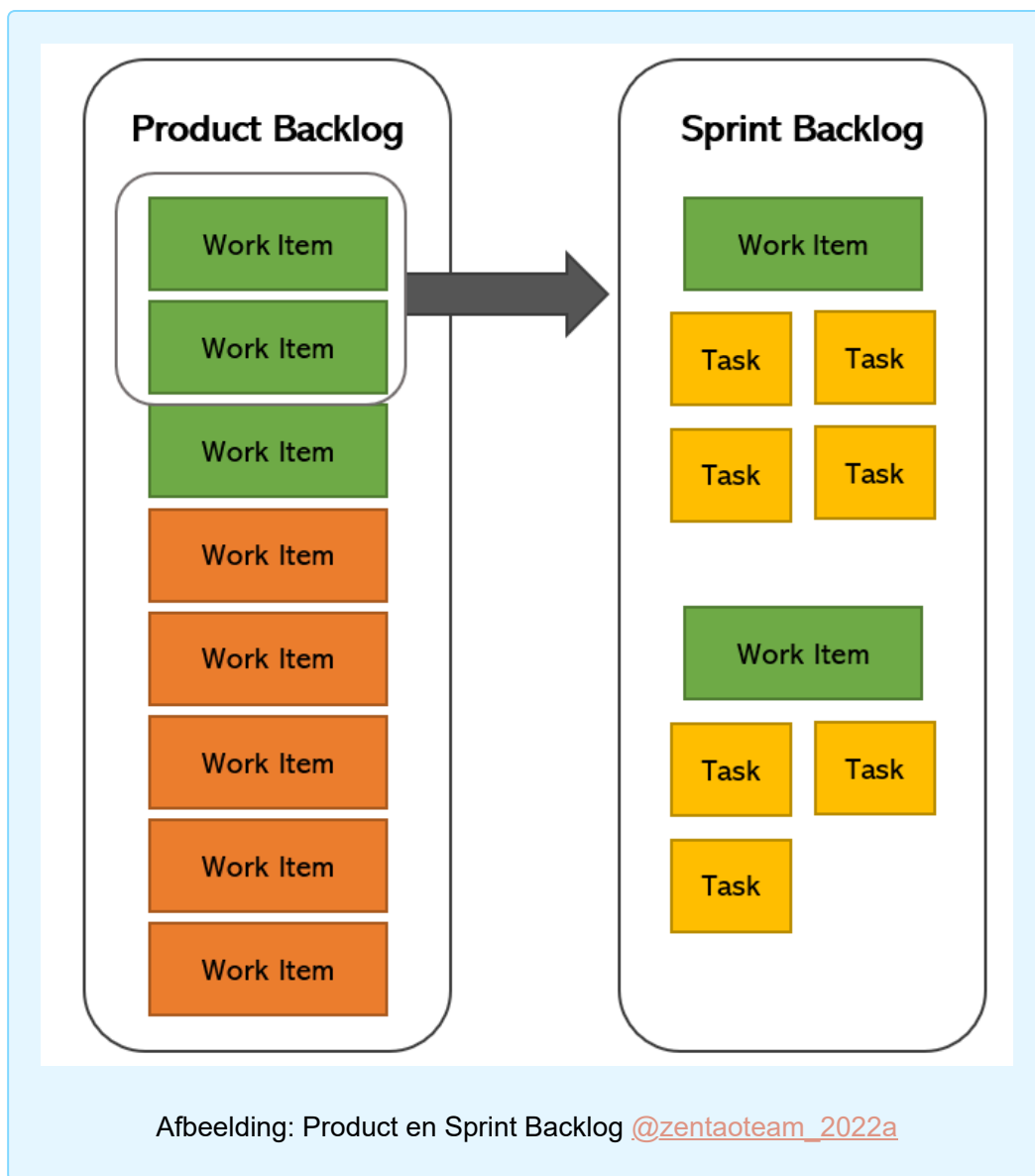
[@raghuprasad_2019](#)

Tijdens de Sprint Planning plannen we de [Sprint Backlog](#). Meer bepaald gaat het team dan beslissen welke [Product Backlog items](#) (PBI's) ze aanpakt tijdens de volgende Sprint. Ze nemen items uit de Product Backlog die ze plaatsen in de [Sprint Backlog](#). Hier kiezen ze, naargelang de belangrijkheid en rekening houdend met de capaciteit van het team, de onderwerpen die ze denken af te ronden in 1 Sprint. Belangrijk is hier de capaciteit van het team goed in te schatten. Dat kan door het bekijken van de [Velocity](#) en door rekening te houden met verlofdagen en opleidingen van leden van het team.

DEFINITIE: Sprint Planning

Sprint Planning: het event dat het begin van een Sprint uitmaakt, met een tijdsbeperking van acht uur of minder. Tijdens dit event bepaalt het Scrum Team welke selectie van de [Product Backlog](#) in een Sprint wordt opgenomen en identificeert het Sprint Doel en de ontwikkelactiviteiten ervoor. [@verheyen_2022](#)

Niet alleen de [Sprint Backlog](#) wordt tijdens de Sprint Planning bepaald, er wordt tevens een duidelijk [Sprint Doel](#) vastgelegd.



Sommige teams kiezen bepaalde taken aan bepaalde ontwikkelaars toe te wijzen, maar dit is niet nodig. Het gros van de teams beslist alleen maar over de [Sprint Backlog](#), en laat de teamleden beslissen wie aan welke taak werkt tijdens de uitvoering van de Sprint.

User Stories en Epics

Een **User Story** is een Agile en Scrum werkinstrument dat als doel heeft producten ("software") te beschrijven vanuit het standpunt van een eindgebruiker.

Herinner je dat we in het begin van deze cursus over Increments spraken? Een User Story is het perfecte voorbeeld van een Increment in Scrum.

User Story

User Stories bevatten **het type gebruiker, wat ze willen en waarom**.

We schrijven ze als volgt:

Vorm

"**als** (type gebruiker), **wil ik** (een bepaald doel) **zo dat** (een bepaalde reden)"

voorbeeld:

Voorbeeld

"**Als** een gebruiker, **wil ik** in mijn account kunnen inloggen met een gebruikersnaam en wachtwoord, **zo dat** ik toegang kan krijgen tot mijn accountinformatie"

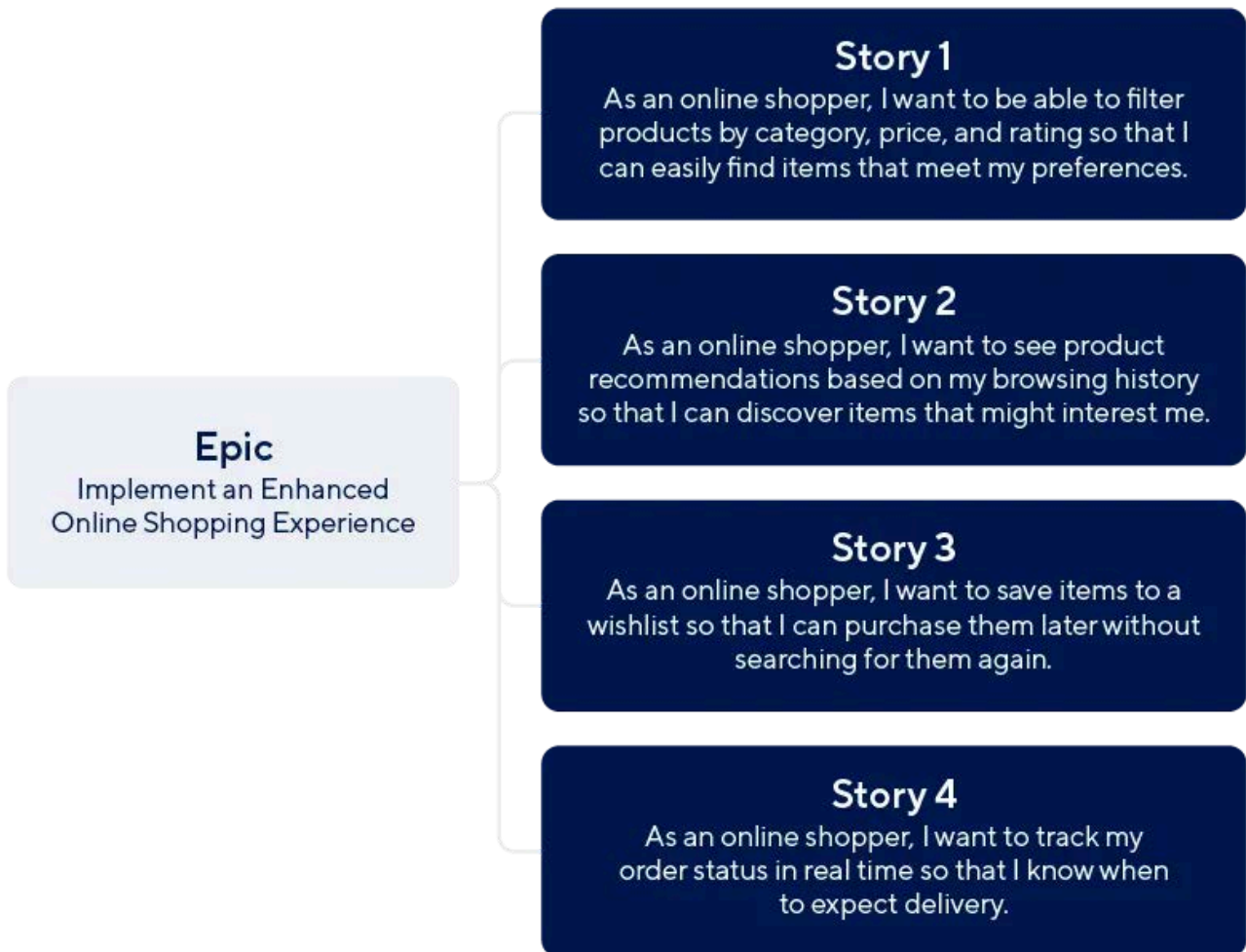
Eindgebruikers zijn geen developers!!!

Je gaat nooit (never, jamais!) Users Stories zien die starten met "Als developer ...", "Als netwerkbeheerder...". User Stories hebben enkel waarde voor een gebruiker. Anders zouden we hen "Developer Stories" noemen

Agile en Scrum laten ontwikkelaars werken aan taken beschreven door User Stories.

Epics, in Agile en Scrum, verwijzen naar grotere blokken werk (grote taken) die we vervolgens verder verdelen in User Stories (kleinere taken).

Epics and User Stories



Smartsheet Inc. © 2024

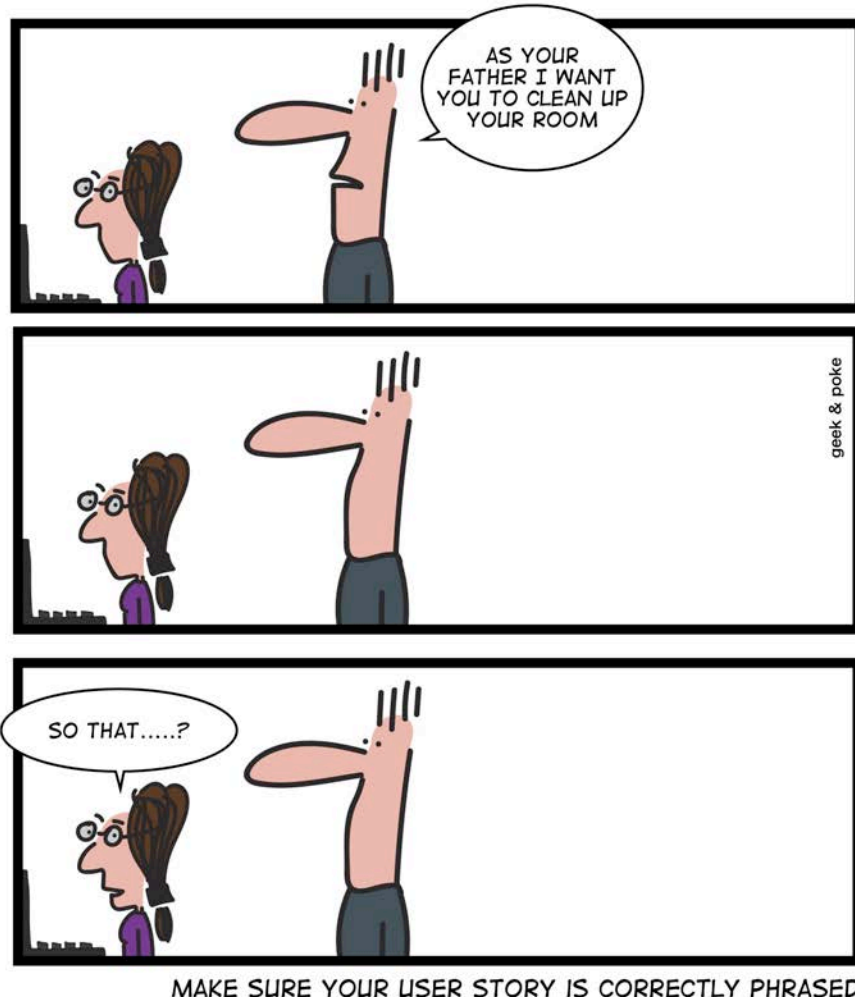
Zowel Epics als Stories zijn voorbeelden van [Product Backlog items](#) (PBI's).

Voorbeelden van User Stories in een Product Backlog

als een	wil ik	zodat
administrator	een lijst zien van leden en bezoekers	ik de sitebezoeken kan monitoren
administrator	nieuwe categorieën kunnen toevoegen	ik leden kan toelaten interessante content te maken
administrator	nieuwe securitygroepen toevoegen	security levels voldoende zijn
administrator	nieuwe trefwoorden toevoegen	inhoud makkelijk te groeperen en te doorzoeken is
administrator	commentaar kunnen verwijderen	beledigende commentaar kan verwijderd worden
administrator	inhoud kunnen tegenhouden	concurrenten en overtreders geen inhoud kunnen versturen

als een	wil ik	zodat
administrator	wil ik de site stijl kunnen aanpassen	de site future-proof is indien de branding van het bedrijf verandert
lid	mijn wachtwoord veranderen	ik veilig kan blijven
lid	mijn contactgegevens aanpassen	administrators mij kunnen contacteren
lid	mijn email voorkeuren kunnen aanpassen	ik niet gebombardeerd word met junk email
lid	inhoud delen op sociale netwerken	ik kan delen wat ik interessant vind
bezoeker	een account aanmaken	ik ledenkorting kan krijgen
bezoeker	kunnen inloggen	ik nieuwe inhoud kan delen
bezoeker	commentaar kunnen invoeren	ik mijn mening kan verkondigen
bezoeker	verbetering voorstellen	ik mijn bijdrage kan leveren aan de bruikbaarheid van de site
bezoeker	administrators contacteren	ik een rechtstreekse vraag kan stellen
bezoeker	de updates van een lid volgen	ik op de hoogte blijf van updates van leden die ik interessant vind
bezoeker	het profiel van een lid zien	ik meer kan weten over een lid

AGILE FAMILIES



MAKE SURE YOUR USER STORY IS CORRECTLY PHRASED

Afbeelding: het is belangrijk je User Stories correct te schrijven

Acceptatie Criteria

Acceptatiecriteria geven een duidelijke beschrijving van wat men verstaat onder "done" (gedaan) in de terminologie van User Stories.

Ze beschrijven de standaarden waartegen iets wordt beoordeeld, de requirements.

☰ Voorbeeld

User story: **Als** een gebruiker van deze website, **wil ik** dat de plaatjes vergroten wanneer ik er met de muis over beweeg, **zo dat** ik onder de indruk ben van de interactiviteit van de

site

☰ Voorbeeld

Acceptatiecriteria: Alle beelden worden 25% groter wanneer men er met de muis over komt.

Agile User Stories (Video)

Product Backlog item

Een **"Product Backlog item"** (PBI, Backlog item of item genoemd) is een eenduidige taak die je moet afronden.

User Stories, Epics, Taken, Bugs of Change Requirements zijn voorbeelden van Product Backlog items.

De [Product Owner](#) verzamelt en prioriteert de [Product Backlog](#) door de dringendste of belangrijkste PBI's bovenaan te plaatsen.

Bijvoorbeeld, wanneer men een website gaat maken van 10 pagina's, kun je elke webpagina als een PBI omschrijven.

Sprint Doel

Het Sprint Doel (of de Sprint Goal) is een korte zin van een of twee regels die beschrijft wat het Team wil bereiken in deze [Sprint](#). Deze zin wordt door de [Product Owner](#) en het Team gezamenlijk opgesteld.

☰ Voorbeeld

- Implementeer een 'winkelwagen'-functionaliteit met toevoegen, verwijderen en update mogelijkheid.
- Ontwikkel een 'check out'-mogelijkheid: betalen voor de order, verzending etc.

Het Sprint Doel is handig om [Stakeholders](#) op de hoogte te houden. Stakeholders hebben geen behoefte aan alle gedetailleerde items; het doel van de komende Sprint te weten is al genoeg.

✎ DEFINITIE: Sprint Doel

Sprint Doel: een korte omschrijving van de overkoepelende doelstelling van een Sprint.

[@verheyen_2022](#)

Het succes van deze Sprint wordt beoordeeld in een later event, de [Sprint Review](#). Het opgeleverde wordt vergeleken met het Sprint Doel. In de [Sprint Review](#) kijkt men niet naar de specifieke items, maar beoordeelt men of het doel is behaald.

[@agile4all_2019](#)

Sprint Backlog

Een "**Sprint Backlog**" is de taak (of de taken) die een team hoopt af te ronden tijdens een bepaalde Sprint.

Een Sprint Backlog is een lijst met User Stories die tijdens een Sprint moeten worden voltooid. Het team schat de User Stories in Storypunten en werkt eraan om ze allemaal af te ronden.

DEFINITIE: Sprint Backlog

Sprint Backlog: De Sprint Backlog is samengesteld uit het Sprint Doel (waarom), de set van Product Backlog items geselecteerd voor de Sprint (wat) en een uitvoerbaar plan voor het opleveren van het Increment (hoe).

Voorbeeld van User Stories uitschrijven voor een Sprint

Stel je voor dat je een app ontwikkelt genaamd "StudieTracker". Hier zijn enkele User Stories die klein genoeg zijn om in een korte Sprint op te nemen en uit te werken:

User Story: Inloggen

"Als student wil ik kunnen inloggen met mijn studentnummer en wachtwoord, zodat ik toegang krijg tot mijn persoonlijke studiegegevens."

Acceptatiecriteria:

- Er is een inlogscherm met velden voor studentnummer en wachtwoord
- Bij correct inloggen word je doorgestuurd naar het hoofdscherm
- Bij foutieve gegevens krijg je een duidelijke foutmelding

User Story: Overzicht studiepunten

"Als student wil ik een overzicht zien van mijn totaal behaalde studiepunten, zodat ik in één oogopslag weet hoe ver ik ben in mijn studie."

Acceptatiecriteria:

- Het hoofdscherm toont het totaal aantal behaalde studiepunten
- Er is een duidelijke weergave van het percentage voltooide studiepunten
- De informatie wordt automatisch bijgewerkt na het inloggen

☰ User Story: Lijst van Opleidingsonderdelen

"Als student wil ik een lijst kunnen zien van al mijn OLODs, zodat ik een overzicht heb van mijn studieprogramma."

Acceptatiecriteria:

- Er is een aparte pagina met een lijst van alle Opleidingsonderdelen
- Per OLOD wordt de naam en het aantal studiepunten getoond
- De OLODs zijn gesorteerd op studiejaar

☰ User Story: OLOD details

"Als student wil ik op een OLOD kunnen klikken om meer details te zien, zodat ik specifieke informatie over dat vak kan inzien."

Acceptatiecriteria:

- Bij het klikken op een OLOD opent een detailpagina
- De detailpagina toont de naam, code, aantal studiepunten en lector(en)
- Er is een indicator die laat zien of het OLOD behaald is of niet

☰ User Story: Voortgangsbalk

"Als student wil ik een visuele voortgangsbalk zien, zodat ik in één oogopslag kan zien hoe ver ik ben in mijn studie."

Acceptatiecriteria:

- Op het hoofdscherm is een voortgangsbalk zichtbaar

- De balk toont het percentage voltooide studiepunten
- De kleur van de balk verandert naarmate je verder bent (bijv. van rood naar groen)

Deze User Stories zijn klein genoeg om binnen een korte Sprint te implementeren. Ze bouwen op elkaar voort en geven samen een goed beeld van de kernfunctionaliteit van de app.

Je zou deze stories kunnen prioriteren in je Product Backlog. Bijvoorbeeld, de inlogfunctie (Story 1) en het overzicht van studiepunten (Story 2) zouden hoge prioriteit kunnen krijgen als minimaal werkbaar product (MVP). De andere stories kunnen dan in volgende Sprints worden opgepakt.

Verschil tussen Product Backlog en Sprint Backlog

Ten slotte, voor wie het nog niet duidelijk is, hier een overzicht van de verschillen tussen de Product Backlog en de Sprint Backlog

Product Backlog	Sprint Backlog
De nodige lijst van alle Product Backlog items zodat het eindproduct kan ontwikkeld worden	De lijst van items, gekozen uit de Product Backlog, die moeten afgewerkt worden om te bekomen dat de Sprint afgerond is
De Product Owner is verantwoordelijk voor het verzamelen van de Product Backlog items en prioriteert en verfijnt ze	Het team is verantwoordelijk voor het maken van de Sprint Backlog en die te verwerken om de Sprint af te ronden
De Product Backlog is bepaald door het volledig doel van het product	De Sprint Backlog is bepaald door het Sprint Doel van een specifieke Sprint
Kan veranderen afhankelijk van de visie van de klant	De Sprint Backlog mag uitzonderlijk, in het geval dat het Sprint Doel dezelfde blijft, veranderen
Alle productkenmerken en storypunten zijn toegewezen aan de individuele User Stories	De Sprint Backlog gedraagt zich als een todo list voor elke Sprint. De Ontwikkelaar verdeelt de User Stories in taken zodat de ingeschatte tijd om ze af te ronden kan berekend worden.
De Product Backlog bestaat zolang het hele product wordt ontwikkeld en moet altijd onderhouden worden	Elke nieuwe Sprint heeft een nieuwe Sprint Backlog, die dan ook eindigt bij het beëindigen van de Sprint

Daily Standups



[@vanderwardt_2017](#)

De dagelijkse meetings voor de werknemers die aan een project samenwerken noemen we de "Daily Standups". De bedoeling is dat deze maar 5 tot 15 minuten duren. Gebruikelijk staan alle aanwezigen recht. Dit heeft als doel de deelnemers aan te moedigen de vergadering niet nodeloos te rekken. De duurtijd van 15 minuten is een maximum!

DEFINITIE: Daily Standup

Daily Standup: Een dagelijkse meeting van maximum 15 minuten waar elke deelnemer de groep de antwoorden op de volgende 3 vragen vertelt:

1. Wat heb ik gisteren bereikt?
2. Wat ga ik vandaag doen?
3. Welke mogelijke obstakels belemmeren mijn vooruitgang?

Daily Standups helpen te verzekeren dat mogelijke belemmeringen (zie het stuk over [Obstake!](#)) om je werk af te ronden snel geïdentificeerd en vlot weggewerkt raken.

VOORBEELD: Daily Standup

Stel je voor dat je in een fastfoodrestaurant werkt. Elke shift begint met een korte standup van 15 minuten. Iedereen vertelt wat ze gisteren hebben gedaan, wat ze vandaag gaan doen, en of er obstakels zijn. Bijvoorbeeld: "Gisteren heb ik de frietketels schoongemaakt,

vandaag ga ik de voorraad bijvullen, en ik merk dat we bijna door de hamburgerbroodjes heen zijn."

Sprint Review

De Sprint Review is bedoeld als het officiële moment tijdens elke Sprint waarin het Scrum Team aan de [Stakeholders](#) (zie iets verder) het gemaakte (deel)product toont. De Stakeholders geven hun terugkoppeling op het (deel)product dat gedemonstreerd is.

Daarnaast tonen we alleen de User Stories die daadwerkelijk zijn afgerond tijdens de Sprint. Items die niet (helemaal) klaar raakten tonen we niet. Dit zou een verkeerde indruk geven van de hoeveelheid waarde die opgeleverd is.

Het laatste onderdeel van de Sprint Review is het bespreken van de [Product Backlog](#) door de [Product Owner](#). Deze neemt de [Stakeholders](#) mee in de stand van zaken. Welke items hebben de hoogste prioriteit en gaan we tijdens de volgende Sprint oppakken? Welke nieuwe items bevat de [Product Backlog](#)?

[@agilescrumgroup_2018](#)

DEFINITIE: Sprint Review

Sprint Review: het event dat het einde van de ontwikkelactiviteiten van een Sprint uitmaakt, met een tijdsbeperking van vier uur of minder. Tijdens dit event bespreken het Scrum Team en uitgenodigde belanghebbenden de resultaten van de [Sprint](#), de huidige inhoud van de [Product Backlog](#) en de voortgang op productniveau. Het doel is om potentieel belangrijk werk voor de nabije, of verdere, toekomst te identificeren. Het resultaat is een bijgewerkte Product Backlog. [@verheyen_2022](#)

Stakeholders

Wat is een stakeholder?

Stakeholders beschouwen we als alle partijen die interesse hebben in een op te leveren product of project. Ze beïnvloeden het project of het project beïnvloedt hen, ze hebben (in)direct invloed en interesse in het Scrum-project.

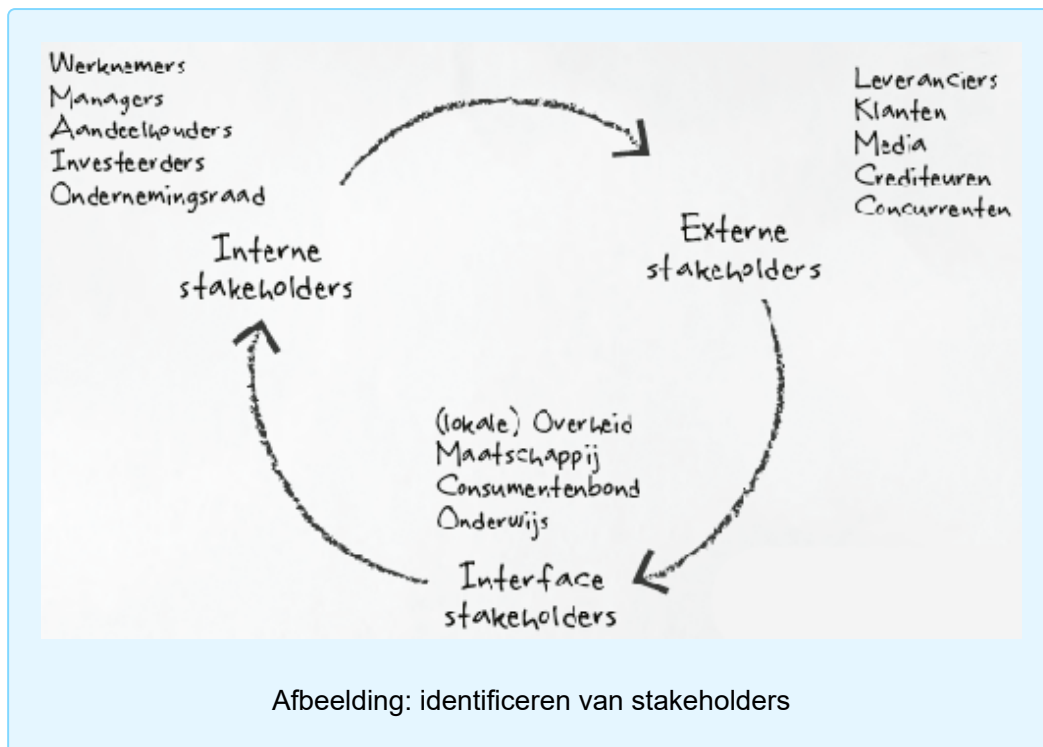
Door deze afhankelijkheid zijn ze in staat het project te maken of te breken. Aan de ene kant beschouwen we als stakeholders de mensen die dicht bij je Scrum-project staan, zoals je belangrijkste klanten, managers en interne opdrachtgevers. Aan de andere kant zijn er de personen die verder van je af staan, zoals de CEO of de Board of Directors.

DEFINITIE: Stakeholder

Stakeholder: een persoon van buiten het Scrum Team met een specifiek belang in het product of kennis ervan die noodzakelijk is voor de verdere ontwikkeling van het product.

[@verheyen_2022](#)

Stakeholders identificeren



Het identificeren van stakeholders kan je als [Product Owner](#) alleen doen, maar je kan hierbij evengoed de hulp van de rest van het Scrum Team vragen. We kennen drie categorieën: interne, externe en interface stakeholders:

1. **Interne stakeholders:** deze willen vooral dat het project winstgevend en/of efficiënt verloopt. Bijvoorbeeld de afdelingen binnen de organisatie, zoals logistiek, marketing, finance, HR.
2. **Externe stakeholders:** deze zitten op behoefte-invulling van het project en hebben een emotioneel belang. Het betreft bijvoorbeeld de eindgebruiker, de consument, de betalende klant.
3. **Interface stakeholders:** deze hebben mogelijk invloed door wet- en regelgeving, bijvoorbeeld de overheid, maatschappij, onderwijs.

[@vanlier_2018](#)

Sprint Retrospective

"Retrospect" (terugblik) betekent het reviseren of bevragen van voorbije gebeurtenissen of tijdsperiodes.

Een **Sprint Retrospective** (of kort: een Sprint Retro) is de meeting op het einde van een Sprint waar het team bepaalt wat men kan wijzigen zodat de volgende Sprint meer productief wordt. Sprint Retrospectives verzamelen feedback en geven informatie over hoe de voortgang van een project verloopt.

De Sprint Retrospective is een meeting voor het volledige Scrum Team. In de praktijk kiezen sommige teams ervoor om eerst intern (alleen Developers) te reflecteren voordat de volledige groep samenkomt.

DEFINITIE: Sprint Retrospective

Sprint Retrospective: het event dat het einde van een Sprint uitmaakt, met een tijdsbeperking van drie uur of minder. Tijdens dit event kijkt het Scrum Team terug op de aflopende Sprint, met als doel om op basis van de ervaringen in de huidige Sprint het proces en de manier van werken, in brede zin, voor de volgende Sprint te bepalen.

[@verheyen_2022](#)

Hier enkele voorbeelden van typische vragen die men kan vragen tijdens een Sprint Retrospective:

- wat waren onze succesvolle acties gedurende deze Sprint?
- wat leerden we van deze Sprint?
- wat gaan we blijven doen in de toekomstige Sprints?
- wat gaan we beter doen in de toekomstige Sprints?
- traden er problemen/fouten op?
- heeft er nog iemand een vraag?
- is er nog iets niet duidelijk voor iemand?

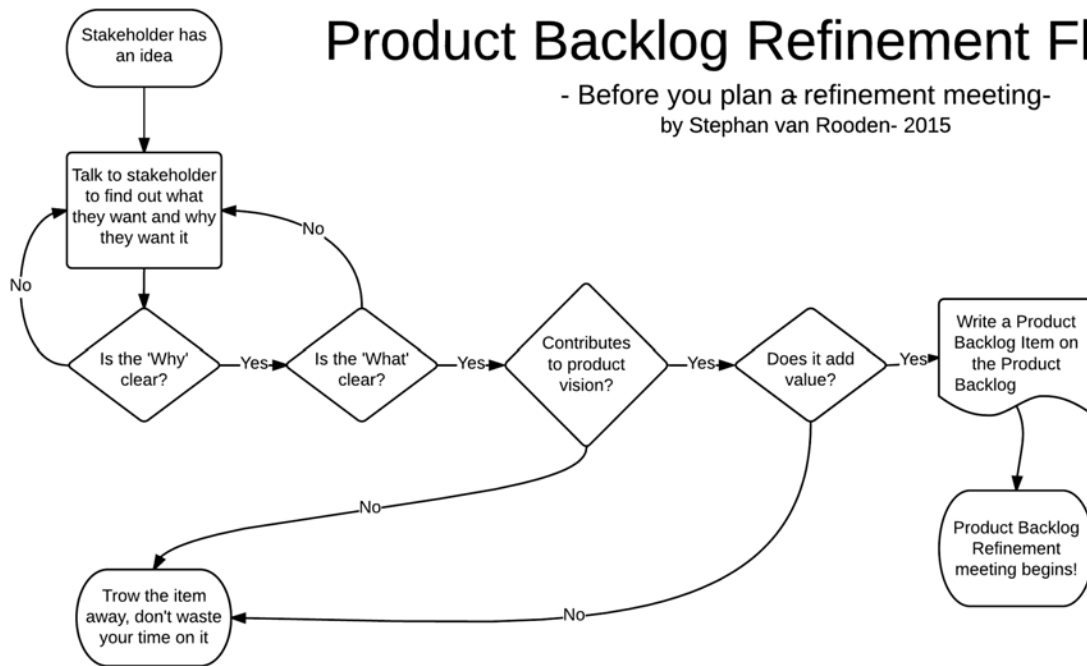
VOORBEELD: Retrospective van een groepsproject

Na een groeps PE voor een vak kom je samen om te bespreken wat goed ging en wat beter kon. Je bespreekt bijvoorbeeld de communicatie binnen de groep, de taakverdeling, en hoe jullie omgingen met deadlines. Jullie maken concrete afspraken voor verbetering in de volgende PE.

Product Backlog Refinement

Product Backlog Refinement Flowchart

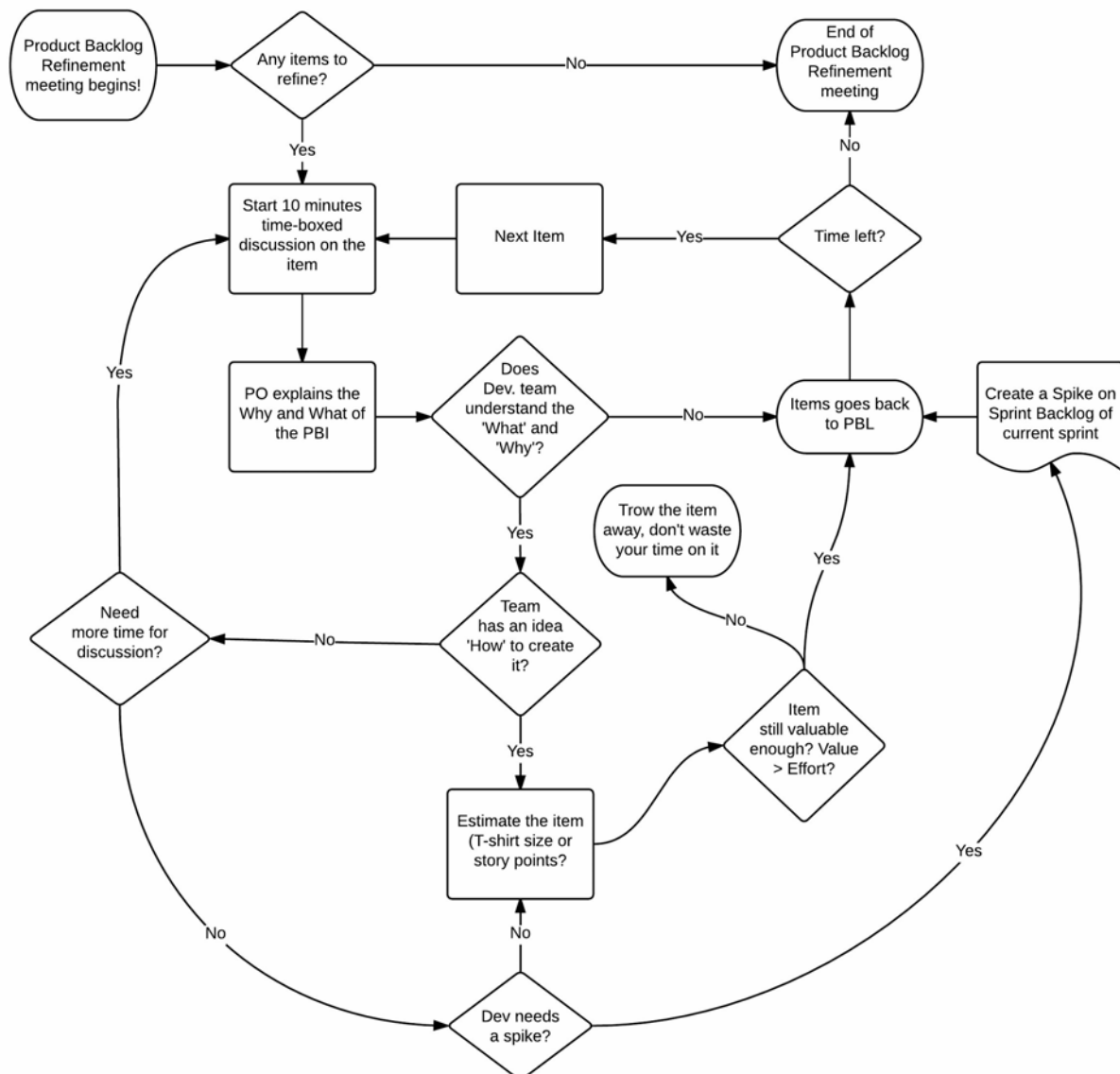
- Before you plan a refinement meeting-
by Stephan van Rooden- 2015



[@vanrooden_2015](#)

Product Backlog Refinement Flowchart

- How to do Product Backlog Refinement -
by Stephan van Rooden- 2015



[@vanrooden_2015a](#)

Product Backlog Refinement is een meeting gedurende de Sprint waarin we items van de Product Backlog bespreken en verduidelijken. We proberen in te schatten hoeveel **Storypunten** er voor elke PBI (**Product Backlog item**) nodig zijn om deze af te ronden en we geven de taken een prioriteit.

Backlog Refinement noemt men ook wel "Backlog Grooming" of "Story Time".

Product Backlog Refinement is GEEN officieel Scrum Event! Maar omdat veel teams het toch toepassen geven we het hier ter verduidelijking.

DEFINITIE: Product Backlog Refinement

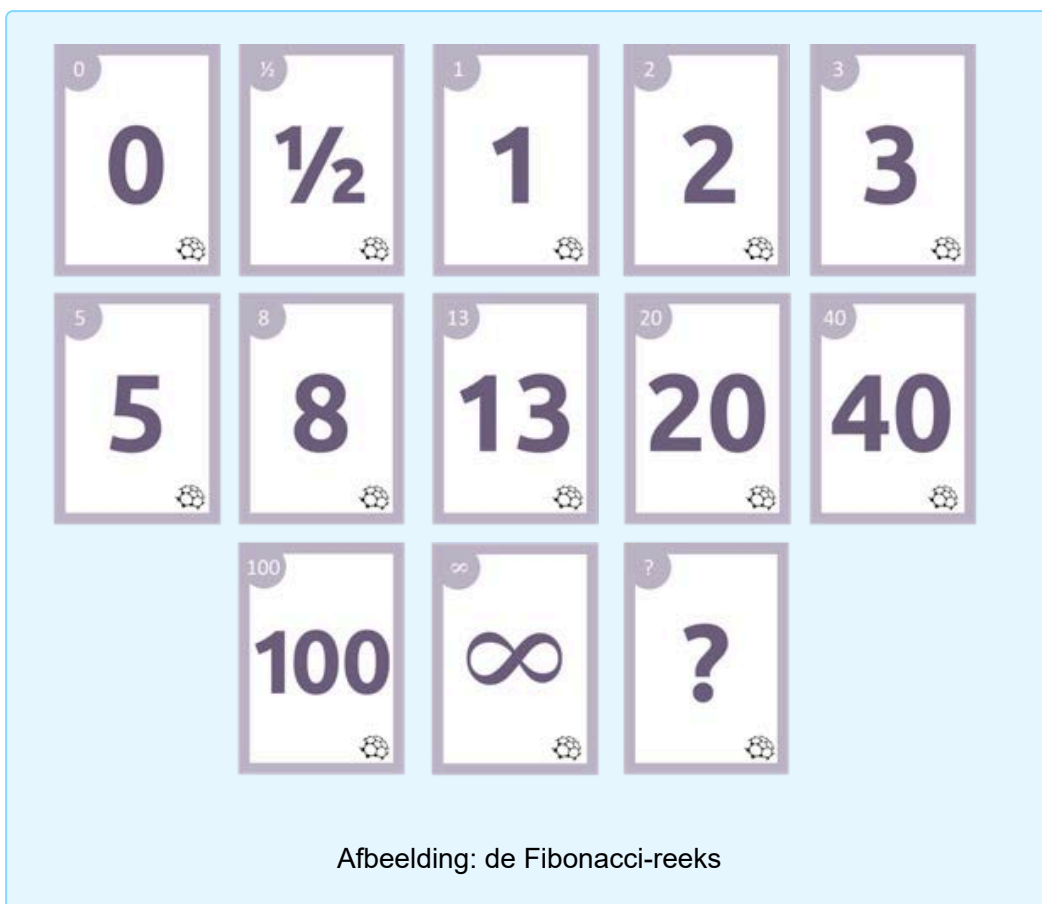
Product Backlog refinement: de regelmatige activiteit om gradueel verfijndere inzichten

te creëren in de [Product Backlog](#). @verheyen_2022

Een belangrijk aspect hierin waarover je zeker moet leren is de inschatting van moeilijkheid een taak af te ronden. De twee gebruikelijkste zijn "T-shirt grootte" en "Fibonacci-reeks". Beide methodes duiden aan hoeveel werk er nodig is voor een bepaalde taak.

De "T-shirt grootte"-toepassing betreft het toewijzen van een T-shirtmaat, welke de moeilijkheid van uitvoering aangeeft, aan elke taak. Gebruikelijk zie je small, medium, large en extra-large.

De "Fibonacci-reeks"-uitvoering betreft het toewijzen van een nummer, welke de moeilijkheid aangeeft, aan elke taak. Deze waardes komen van de bekende wiskundige reeks nummers, ooit, eeuwen geleden, ontwikkeld door de Italiaanse wiskundige Fibonacci. Ze vormen een reeks nummers waar elk nummer de som is van de vorige twee nummers:



1, 2, 3, 5, 8, 13, enz.

Hoe hoger het getal, hoe moeilijker de taak is.

Wanneer een bepaalde taak buiten de aanvaardbare range van de T-shirtgrootte of Fibonacci-reeks valt, wordt bepaald dat ze de taak best opsplitsen in kleinere, behapbare deeltaken. We bekijken dit verderop in detail bij de tekst over [Storypunten](#).

Scrum Artefacten

De artefacten van Scrum vertegenwoordigen werk of waarde. Ze zijn ontworpen voor maximale transparantie van de belangrijkste informatie. Dankzij deze transparantie heeft iedereen, die deze artefacten inspecteert, dezelfde basis voor aanpassing.

Elk artefact bevat een commitment om ervoor te zorgen dat het informatie geeft die de transparantie en de focus verhoogt, waaraan de vooruitgang kan worden gemeten:

- Voor de [Product Backlog](#) is dit het Product Doel.
- Voor de [Sprint Backlog](#) is dit het [Sprint Doel](#).
- Voor het [Increment](#) is dit de [Definition of Done](#).

Deze commitments bestaan om empirisme en de Scrum-waarden te versterken voor het Scrum-team en zijn belanghebbenden.

What is Scrum? (Video)

Hulpmiddelen en tools tijdens de Sprint

Storypunten



We raden altijd aan om met storypunten te werken.

Probeer tijdsindicaties te vermijden. Waar het ene teamlid vier uur voor een User Story nodig heeft, kost hetzelfde klusje een ander teamlid zes uur.

Door een eenheid te creëren die voor het hele team hetzelfde is, voorkomen we discussies over tijd en teamleden. In plaats van te kijken hoe lang een User Story duurt, richten we ons op de omvang van de klus.

Wat betekent de relatieve waarde?

De storypunten geven een beeld van de omvang van een User Story. Een User Story van 3 punten is groter dan een story van 1 punt, en kleiner dan een story van 5.

Wat schat je in?

Om het aantal punten van een User Story te bepalen, kijkt het team naar drie factoren die aangeven hoeveel moeite het kost om de User Story af te ronden.

Hoeveelheid

Hoeveel werk is het om de User Story volledig(!) af te ronden, volgens de [Definition of Done](#)?

Complexiteit

Hoe ingewikkeld is het om de User Story te maken? Is het makkelijk, of juist uitdagend?

Onzekerheid

Hoe groot is de kans dat we verrassingen tegenkomen die we vooraf niet hadden voorspeld?

Samen bepalen deze factoren de hoeveelheid punten van een User Story.

Bij een lage complexiteit, risico en moeite, krijgt de story een laag storypunt.

Pizza-punten

De Pizza-analogie voor Storypunten

Stel dat je met een bende vrienden verschillende soorten pizza's wil bestellen. De vriendengroep wil de berg pizza's inschatten, maar in plaats van over calorieën of grammen te praten, gebruiken jullie de grootte en moeilijkheid (veel kaas/olijven of pikant) van de pizza's als criterium.

- Een kleine margherita pizza noemen jullie een '1'
- Een medium quattro formaggi is een '3'
- Een grote pepperoni met extra kaas is een '5'
- En een familiepizza met alles erop en eraan is een '8'

Deze nummers zijn jullie 'pizza-punten'. Ze zeggen niets over de exacte grootte of het aantal calorieën, maar geven wel een idee van hoe 'groot' of 'complex' elke pizza is om op te eten.

Leen kan misschien twee '3'-pizza's op, terwijl Stef met moeite één '5'-pizza kan verwerken omwille van de gesmolten kaas die hem in het verleden al de das omdeed. Arnaud, die net heeft gesport, kan probleemloos een '8' verorberen.

Iedereen heeft een andere "capaciteit", maar iedereen begrijpt wat bedoeld wordt met een '3' of een '5' pizza.

Storypunten in Scrum werken op dezelfde manier:

- De punten geven een relatieve inschatting van de complexiteit of grootte van een taak.
- Een '1' is een kleine, eenvoudige taak, terwijl een '8' een grote, risicovolle en complexe taak is.
- Verschillende teamleden kunnen verschillende hoeveelheden werk aan, net zoals de vrienden verschillende hoeveelheden pizza kunnen eten.
- Het Agile-team leert na verloop van tijd hoeveel 'punten' ze gezamenlijk aankunnen in een Sprint, net zoals de vriendengroep leert hoeveel 'pizza-punten' ze aankan in één avond.

Het voordeel van dit systeem is dat je niet hoeft te discussiëren over exacte uren of moeilijkheidsgraden. Je kunt snel en intuïtief taken vergelijken en inschatten, net zoals je pizza's kunt vergelijken zonder over precieze afmetingen of ingrediënten te praten.

Hoe kun je er zelf mee starten?

Om met storypunten te starten, moet je team eerst een **referentie-story bepalen** waarmee je alle andere stories kunt vergelijken. Kies hiervoor een redelijk complexe User Story die een zo hoog mogelijk aantal disciplines van je team omvat. Geef deze story 5 of 8 punten. Dan heeft je team nog genoeg ruimte om stappen naar boven en beneden te nemen. Je kunt de referentie-story tijdens [Product Backlog Refinements](#) gebruiken om te bepalen of een andere User Story kleiner of groter is, en welke waarde je hieraan kunt geven.

Samengevat

Storypunten zijn een relatieve waarde die je samen met je Scrum Team creëert. Ze bieden inzicht in de moeite die het kost om een User Story te maken. De Product Owner kan afwegen om een bepaald item al dan niet te prioriteren. En het team weet hoeveel storypunten het per Sprint kan oppakken.

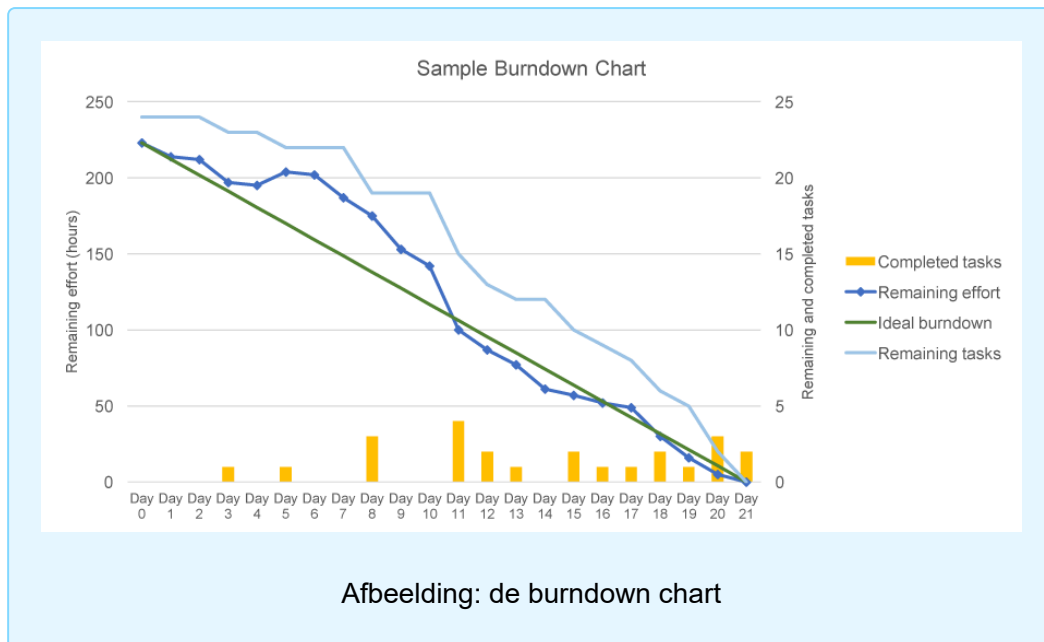
[@vriellink_2020](#)

What are Story Points (Video)

De Burndown Chart

Tijdens een Sprint toont een burndown chart voor het project hoeveel werkuren er op dagbasis nog overschieten. De dagen zie je van links naar rechts onderaan de grafiek en de uren zie je aan de linkerzijde.

Kijkend naar de Burndown Chart kun je zien of een project nog "on target" is, voorloopt of achterophinkt.



Meerdere mogelijke varianten, of manieren waarop je een burndown chart kan opstellen bestaan. Daar je in Scrum eerder in storypunten inschat, kiest men dan de verticale as af te zetten tegen de overgebleven storypunten i.p.v. de uren.

☰ VOORBEELD: Burndown Chart in de werkelijke wereld

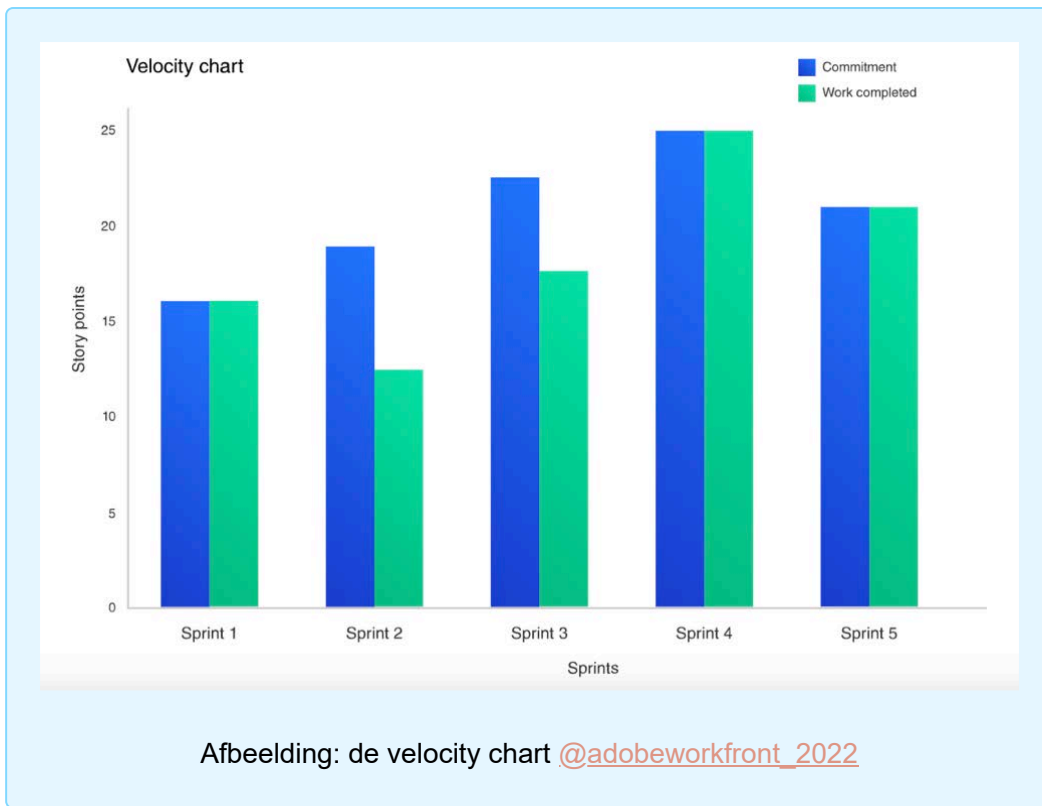
Stel je voor dat je een eindwerk schrijft. Je hebt 8 weken (je Sprint) en je moet 15.000 woorden schrijven (je totale werk). Je Burndown Chart zou kunnen laten zien hoeveel woorden je elke dag schrijft en of je op schema ligt.

How to use The Sprint Burndown (Video)

Velocity

Velocity is de snelheid waartegen teams werk afronden. Meer bepaald, de som van de storypunten van de items van de backlog die het team succesvol kan afronden in 1 Sprint.

Eenmaal men de velocity kent na een aantal Sprints, kan men deze waarde gebruiken om het plannen van projecten te helpen en te voorspellen welke opleverdatum te voorzien.



DEFINITIE: Velocity

Velocity: een veelgebruikte aanduiding van de gemiddelde hoeveelheid [Product Backlog](#) die een specifiek team met zijn specifieke samenstelling tijdens een Sprint omgezet heeft in één of meerdere Incrementen. [@verheyen_2022](#)

Definition of Done

Een Definition of Done geeft een duidelijke omschrijving van hoe een opgeleverde Product Backlog item er uit moet zien. Het is een manier om de kwaliteit van opgeleverde producten binnen [Scrum](#) te borgen. Het zorgt voor transparantie en duidelijkheid. Het ontwikkelteam geeft een commitment op het volgen van de Definition of Done.

Een Definition of Done is een checklist met activiteiten die je afvinkt voor iedere User Story. **Dat betekent dat het activiteiten zijn die voor iedere User Story van toepassing zijn.**

Je bereikt onder andere het volgende met een goede Definition of Done:

1. Overeenstemming tussen het ontwikkelteam en de [Product Owner](#) over wanneer een User Story daadwerkelijk klaar is. Heldere verwachtingen bepalen de basis voor goede samenwerking.
2. Het verhogen van de snelheid waarmee werk wordt opgeleverd. Doordat je een algemeen geldende checklist hebt die je kan afvinken kun je snel werk van hoge kwaliteit leveren.

DEFINITIE: Definition of Done

Definition of Done: de verzameling kwaliteiten, verwachtingen en criteria waaraan een Increment moet voldoen om te kunnen worden vrijgegeven naar de eindgebruikers – met respect voor overkoepelende organisatierichtlijnen indien beschikbaar. [@verheyen_2022](#)

De Definition of Done wordt tijdens de start van iedere [Sprint](#) in de [Sprint Planning](#) herhaald. Daarnaast wordt de Definition of Done aangescherpt op basis van voortschrijdend inzicht.

Voorbeeld van een DoD (Definition of Done) voor een IT-project

- Alle code is geschreven (alle 'to do' items in de code zijn afgewerkt).
- Relevante gebruikersdocumentatie is gemaakt en beschikbaar gesteld.
- Er is een installatiehandleiding voor de installatie van de software.
- Alle functionele testen zijn gedraaid.
- Code heeft een peer review ondergaan.

[@agilescrumgroup_2017](#)

Voorbeeld van een DoD voor een niet-IT-project

- Alle to-do's voor de story zijn afgerond.
- Een ander teamlid heeft feedback gegeven op het werk.
- Er is een spellingscheck gedaan.
- Het werk is gecontroleerd op huisstijl.
- Het werk is gecontroleerd op *design* en *brand values*.
- Het werk is gecontroleerd in het kader van compliance.
- Er is feedback van eindgebruikers gevraagd op het opgeleverde product.

Wat de inhoud van *Definition of Done* is, bepaalt het team dus helemaal zelf.

Het verschil tussen Acceptatiecriteria en de DoD



Veel studenten gooien de termen Acceptatie Criteria en Definition of Done door elkaar. Iets doet mensen twijfelen, anderen leggen het fout uit of interpreteren ze anders, maar het zijn duidelijk **totaal** verschillende termen.

Definition of Done (DoD):

- Is een algemene checklist voor alle taken of user stories in een project.
- Bevat standaard kwaliteitseisen die voor elk stuk werk gelden.
- Wordt door het hele team gebruikt.
- Voorbeeld: code review gedaan, tests geschreven, documentatie bijgewerkt.

Acceptatiecriteria:

- Zijn specifiek voor één bepaalde taak of user story.
- Beschrijven de exacte vereisten waaraan die specifieke taak moet voldoen.
- Worden vaak door de Product Owner opgesteld.
- Voorbeeld: "De gebruiker kan inloggen met e-mail en wachtwoord".

⚠ **VERSCHIL**

- **De Definition of Done** is als een algemene checklist voor **alles** wat het team doet.
- **Acceptatiecriteria** zijn als een specifieke takenlijst voor **één bepaald** onderdeel (vb. een Story heeft zijn eigen criteria).

Agile Projectmanagement

Het cruciale verschil tussen de **Definition of Done** en **Acceptatiecriteria**

Definition of Done (DoD)

DE UNIVERSELE CHECKLIST

Een lijst met generieke kwaliteitseisen die geldt voor **ALLE** User Stories binnen het project. Het garandeert dat het werk klaar is om naar productie te gaan.

-  Geldt voor **het hele project**
-  Focus op **kwaliteit** (intern)
-  Eigendom van het **Development Team**

Acceptatiecriteria (AC)

DE SPECIFIEKE WENSEN

Een lijst met specifieke voorwaarden die horen bij **ÉÉN** specifieke User Story. Het beschrijft wat de gebruiker precies wil kunnen doen.

-  Geldt voor **één story**
-  Focus op **functionaliteit** (extern)
-  Eigendom van de **Product Owner**

VS

Scope	Generiek (Macro) Geldt voor alle tickets/stories.	Specifiek (Micro) Geldt voor één specifieke story.
Doel	Kwaliteitsborging Zorgen dat de software niet breekt.	Klantwens vervullen Doet het wat de gebruiker vroeg?
Wanneer is het af?	Release-waardig Het product is klaar voor productie.	Functionaliteit werkt De knop/functie doet het juiste.
Verandert het?	Statisch Zelden, geldt voor hele projectduur.	Dynamisch Uniek per user story.

Concrete Voorbeelden

Voorbeeld Definition of Done

"Dit checken we bij **ELKE** story voordat we zeggen dat het af is:"

- ✓ Code is reviewed door een collega (Peer Review).

Voorbeeld Acceptatiecriteria

"User Story: Als klant wil ik met creditcard betalen."

- ✓ Gebruiker kan Visa en Mastercard selecteren.

✓ Unit tests zijn geschreven en slagen (Groen licht). ✓ Documentatie is bijgewerkt. ✓ Geen openstaande bugs bekend.	✓ Systeem valideert of kaartnummer 16 cijfers heeft. ✓ Bij succesvolle betaling ziet de gebruiker een bedankpagina. ✓ Bij weigering ziet de gebruiker een duidelijke foutmelding.
---	---

Scrum bord



Binnen Scrum wordt gewerkt met een Scrum-bord (*synoniem: 'takenbord' of 'Kanban-bord'*). Het Scrum-takenbord is bedoeld als hulpmiddel bij het overzichtelijk maken van de stand van zaken binnen een Sprint. De [Sprint Backlog](#) bevat alle taken geselecteerd voor een Sprint. Die taken hangen op post-its in één van de drie kolommen op het bord: To Do, Doing of Done. De belangrijkste taken hangen boven aan het bord. Dit bord geeft in één oogopslag inzicht in de status van een Sprint:

- **To do.** Alle taken die men plant uit te voeren binnen de Sprint.
- **Doing.** De taken binnen de Sprint waar op dat moment aan gewerkt wordt.
- **Done.** Alle beëindigde taken binnen de Sprint.

Het doel is om aan het einde van iedere Sprint alle taken onder de laatste kolom te hebben geplakt. Dat betekent dat alle vooraf bepaalde taken werden uitgevoerd en dat het doel van de Sprint behaald is. Het takenbord is voor het Scrum Team de centrale plaats om de aanpak van het resterende werk af te stemmen. Het takenbord wordt een **Scrum Board** genoemd, en wordt gebruikt tijdens de [Daily Scrums](#).

Wanneer de Sprint is afgelopen en niet alle taken staan onder 'Done' dan plaatsen we de niet beëindigde taken terug in de [Product Backlog](#). De [Product Owner](#) kijkt dan vervolgens of deze taken in de volgende Sprint passen.

[@scrumguide_2022a](#)

Wat hebben we geleerd?



We leerden wat [Agile](#) betekent en dat we daar [incrementeel](#) en [iteratief](#) te werk gaan.

Vervolgens gingen we in detail naar [Scrum](#) kijken, de meest gebruikte uitvoering van Agile.

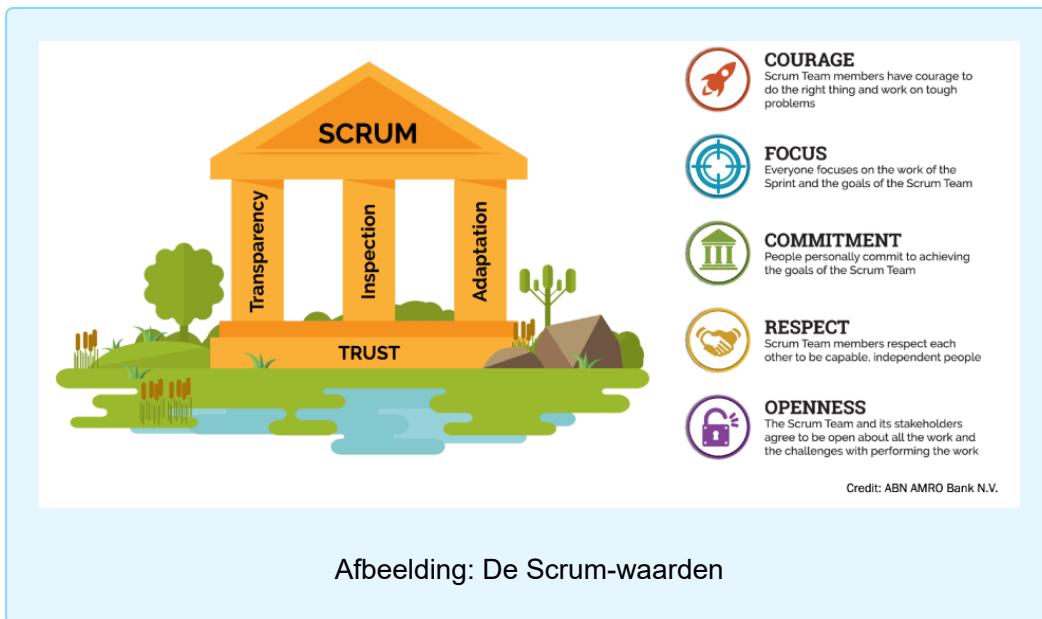
We maakten kennis met [De drie Rollen in Scrum](#) en we leerden wat ze betekenen.

We zagen dat Scrum bestond uit [5 Scrum Events](#), die tijdens een [Sprint](#) plaatshebben.

Verder bekeken we definities, tools en hulpmiddelen die we gebruiken tijdens de uitvoering van Scrum.

- [Product Backlog](#)
- [User Stories en Epics](#)
- [Scrum Artefacten](#)
- [Stakeholders](#)
- [Obstakel](#)
- [Acceptatie Criteria](#)
- [Sprint Doel](#)
- [Sprint Backlog](#)
- [Product Backlog Refinement](#)
- [De Burndown Chart](#)
- [Velocity](#)
- [Storypunten](#)
- [Scrum bord](#)
- [Definition of Done](#)

Wat we zeker nooit gaan vergeten is dat Scrum meer is dan een verzameling regeltjes, rollen en events.



De basis van Scrum is een verzameling van kernwaarden in combinatie met het correct toepassen van empirisch denken.

Belangrijk

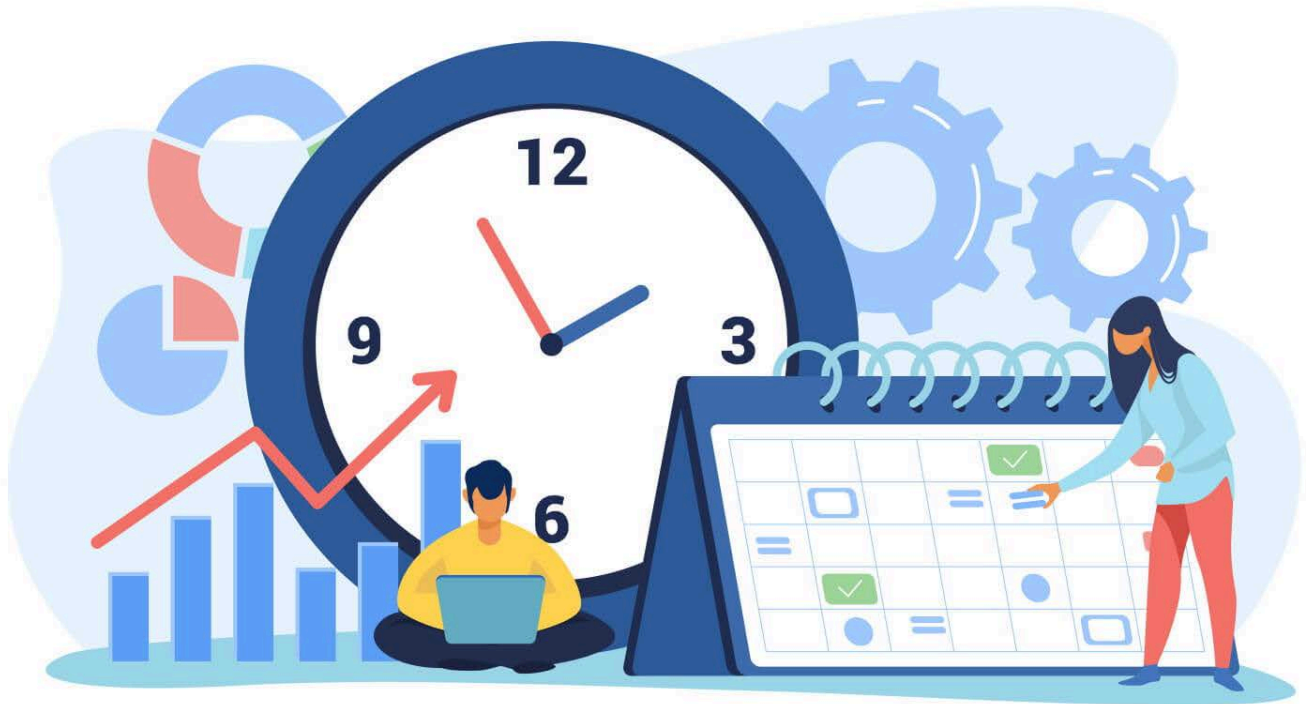
Een degelijke kennis van de theorie is essentieel, maar enkel door toepassing van de basis gaat ze renderen tegen het volledig potentieel.

Als ik aankom op dit punt beginnen mensen vaak somber te worden. Ze beginnen vragen te stellen als: 'In een wereld waarin machines heel snel superslim worden in elke specialistische kennistaak die we ze kunnen geven, waar dienen mensen dan nog voor?' Dat is een belangrijke vraag. En het antwoord is eigenlijk ontzettend opbeurend. Mensen zijn er om nóg menselijker te worden dan we ooit tevoren waren. Menselijker in hoe we werken. Menselijker in wat we leren. En menselijker in hoe we die kennis met elkaar delen.

De TED-methode
Chris Anderson



De zin en onzin van tijdsregistratie in Agile projecten - geen examenleerstof



Het registreren van tijd in een Agile-project is een veelbesproken onderwerp, vooral omdat ontwikkelaars in Scrum Teams zeggen dat het in strijd is met de Agile Principles (*"Werkende software boven allesomvattende documentatie"*).

Toch zijn veel Agile-coaches het erover eens dat de prestaties verbeteren als tijd wordt bijgehouden als individuele activiteit en niet als managementvereiste.

Scrum en Agile-methodologieën zijn flexibel en makkelijk aanpasbaar, waardoor ontwikkelaars meer werk kunnen doen. Het is belangrijk dat je de juiste trackingmetriek kiest om het project sneller te laten verlopen.

Voordelen van tijdsregistratie

Tijdsregistratie in Jira maakt een nauwkeuriger meting van de voortgang van een project mogelijk. Product Owners kunnen zien hoeveel tijd er aan een Story is besteed en hoeveel tijd er nog over is. Het team krijgt door middel van continue uurupdates een vrij nauwkeurige weergave van de voortgang van de Sprint en kan helpen blokkades op te lossen wanneer een probleem langer aanhoudt.

Zo wordt het ook voor ontwikkelaars eenvoudig om historische gegevens op te halen voor toekomstige Sprints, Sprint Planningen, de Sprint Backlog-invulling en Sprint Retrospectives.

Ze weerspiegelen de werkelijkheid, creëren transparantie en helpen het projectbudget te volgen en de winstgevendheid van het project te voorspellen.

Bovendien wordt tijdsregistratie in een bedrijfscontext gebruikt als tool om transparantie naar de klant te garanderen en om facturatie te verantwoorden.

In onze context, het onderwijs, is tijdsregistratie een manier om de bijdrage van elke student zichtbaar te maken.

Nadelen van tijdsregistratie

Ondanks het feit dat iedereen het erover eens is dat tijdsregistratie veel voordelen heeft voor een Scrum Team, kan het systeem de voortgang belemmeren als het niet correct wordt uitgevoerd. Het kan het team schaden als je vraagt de tijd bij te houden als een micromanagementtechniek^[3], wat in strijd is met de Agile-methodologie.

Handmatige urenregistratie vertraagt de ontwikkelaars en vermindert hun productiviteit. Bovendien groeit de kans dat ontwikkelaars uren gaan 'fantaseren' of ze op vage of algemene taken gaan boeken, wat leidt tot verkeerde schattingen, foutieve gegevens en een verlies aan factureerbare uren.

Tijdsregistratie op een Agile manier

Hoewel er geen direct verband is tussen het bijhouden van tijd en de kwaliteit van code, heeft een Product Owner wel verantwoordelijkheden tegenover het management, klanten en belanghebbenden. Wanneer een project langer duurt dan verwacht vanwege wijzigingen in de scope, geven urenstaten een gedetailleerd inzicht in de status van het project en de productiviteit van het Scrum Team.

Belangrijk is de juiste context te koppelen met de juiste registratie.

Praktische handleiding

Scenario 1: Doe je iets in de context van een Story, boek dan je uren op die Story



Agile-teams werken aan User Stories; alles wat je doet is waardevol voor de klant, en die waarden worden uitgeschreven in stories.

In elk van de volgende scenario's boek je je uren onder de betreffende story, met een relevante beschrijving:

1. Je werkt aan acceptatiecriteria voor een story en voegt deze toe aan de beschrijving van de story in Jira.
2. Je doet als developer technisch onderzoek dat nodig is om een story te kunnen beginnen.
3. Je schrijft code voor een story.
4. Je beoordeelt een pull request van een collega-developer voor een story.
5. Je schrijft unittesten voor een story.
6. Je test een story om te zien of deze voldoet aan de acceptatiecriteria.

In principe boek je het merendeel van het werk onder stories.

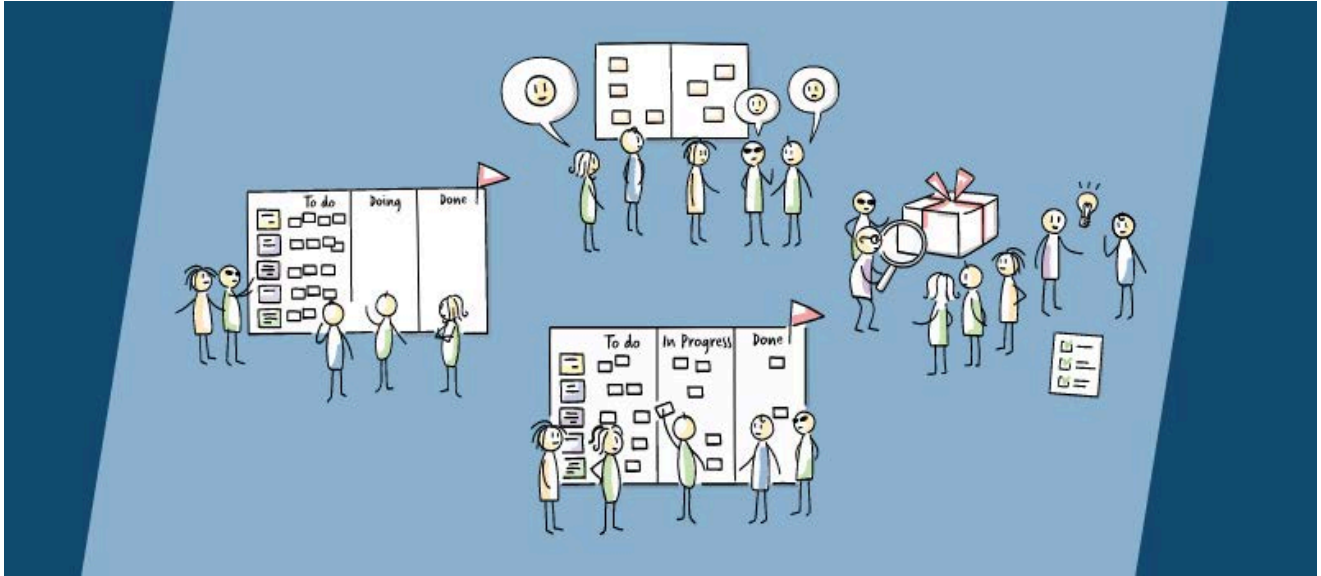
Scenario 2: Ben je bezig met taken die niet strikt in de context van een story vallen, boek dan je uren op die specifieke taak.



Tot dergelijke taken behoren het schrijven van een projectplan, het voorbereiden van een presentatie, het opzetten van de CI/CD-omgeving, het aanmaken van een Git-repository, het werken aan een afstudeer-specifieke taak, enz. Ook het opstellen van documentatie voor een specifieke opdracht valt hieronder. Deze activiteiten hebben, strikt genomen, geen directe waarde voor de klant — niet dat ze waardeloos zijn, want ze helpen de kwaliteit van het product te waarborgen, maar ze voegen geen rechtstreekse functionaliteit toe voor de klant. Daarom kwalificeren ze niet als User Stories, maar als **Taken**.

Let erop dat we geen algemene of vage taken zoals 'testen van de code' of 'vergaderingen' willen zien in de urenregistratie. Dit zijn indicatoren dat er onvoldoende transparantie is in de werking van het team. Dergelijke taken zijn waarschijnlijk onderdelen van een User Story (scenario 1), of een Scrum Event (Scenario 3).

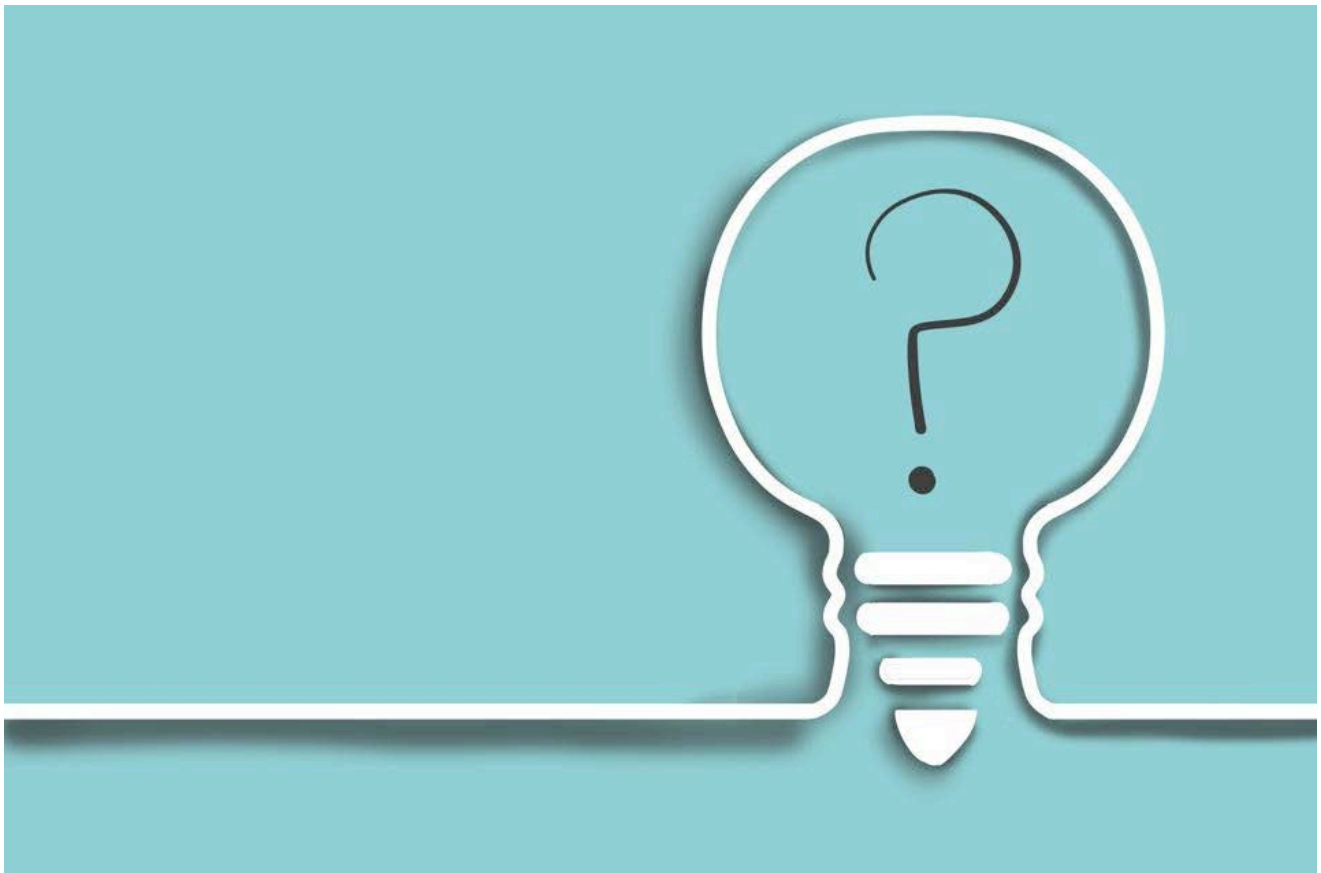
Scenario 3: Wanneer je deelneemt aan een van de Scrum Events, registreer dan de uren onder een taak genaamd 'Scrum Events', met de naam van het specifieke event als beschrijving.



In principe is deze vorm van tijdsregistratie niet nodig, aangezien alle teams de Scrum Events houden en alle teamleden deelnemen aan deze activiteiten. In een volwassen bedrijfscontext gaan teams er vaak van uit dat 20% van de tijd besteed wordt aan de Scrum Events, wat betekent dat slechts 80% van de tijd (8 uur x 80% = 6,4 uur) als productieve werktijd wordt beschouwd.

Omdat we momenteel nog geen volwassen teams zijn, maken we voor iedere Sprint een taak aan genaamd 'Scrum Events' en boeken daar de tijd voor elk afzonderlijk event op.

Boek je uren op Stories/Taken? Of op subtaken onder Stories/Taken?



In principe maakt het niet uit welke van de twee opties je kiest, maar kies zeker per team consistent voor een van de twee opties.

Indien je graag achteraf gegevens wilt hebben over de tijd die is besteed aan analyse, ontwikkeling, en testen, of aan back-end/front-endontwikkeling op storyniveau, **en wanneer je dankzij deze data daadwerkelijk actie onderneemt die de werking van het team optimaliseert**, kan het handig zijn om subtaken te gebruiken. In dat geval registreer je de uren op de subtaak.

Maak je geen analyses, of wordt dit niet gedaan door je bedrijf, docent of welke andere controlerende instantie dan ook, dan is de keuze om uren te registreren met een relevante omschrijving op storyniveau meer dan voldoende en voorkom je onnodig administratief werk voor het team.

Je kunt perfect zien hoeveel tijd het team heeft besteed aan de story. Wees in dat geval duidelijk in je omschrijving en maak goede afspraken.

Bibliografie

- [@adobeworkfront_2022](https://www.workfront.com/project-management/methodologies/agile/velocity): 'What is Velocity in Agile? Charts & Examples | Adobe Workfront' - **Adobe Workfront,(2022)** <https://www.workfront.com/project-management/methodologies/agile/velocity>
- [@agilescrumgroup_2018](https://agilescrumgroup.nl/sprint-Checklist%20Valkuilen): 'De Sprint Review uitgelegd. Waarom deze meeting? (+ Checklist & Valkuilen)' - **Agile Scrum Group,(2018)** [https://agilescrumgroup.nl/sprint-Checklist & Valkuilen\)](https://agilescrumgroup.nl/sprint-Checklist%20Valkuilen)

[review/](#)

- [@agilescrumgroup_2017](#): 'Wat is Definition of Done? Check de uitleg en voorbeelden (IT & non-IT)' - **Agile Scrum Group,(2017)** <https://agilescrumgroup.nl/wat-is-definition-of-done/>
- [@agile4all_2019](#): 'Sprint Planning' - **agile4all,(2019)** <https://www.agile4all.nl/sprint-planning/>
- [@martin_2020](#): 'Incremental Model in SDLC: Use, Advantage & Disadvantage' - **Martin, Matthew(2020)** <https://www.guru99.com/what-is-incremental-model-in-sdlc-advantages-disadvantages.html>
- [@qframe_2021](#): 'Scrum Master, Teambuilder of Agile Coach?' - **qframe,(2021)** <https://qframe.be/2021/06/25/scrum-master-teambuilder-of-agile-coach/>
- [@raghuprasad_2019](#): 'Basics of Sprint Planning - WHAT' - **Raghuprasad,(2019)** <https://agilebatech.com/what-is-sprint-planning/>
- [@scrumguide_2022](#): 'Het Scrumteam: uitleg en tips bij de verantwoordelijkheid | Scrumguide.nl' - **scrumguide,(2022)** <https://scrumguide.nl/scrumteam/>
- [@scrumguide_2022a](#): 'Scrum takenbord | Scrumguide.nl' - **Scrumguide,(2022)** <https://scrumguide.nl/scrumbord/>
- [@stanley.etal_2020](#): 'Project management handbook: simplified Agile, Scrum, and DevOps for beginners' - **Stanley, Jack C; Gross, Erik D; Tech Academy(2020)** -
- [@userstorymap_2022](#): 'How to Be An Effective Product Owner' - **userstorymap,(2022)** <https://www.userstorymap.io/being-an-effective-product-owner/>
- [@vanderwardt_2017](#): 'Daily Stand-up: 5 tips voor een goede meeting (+checklist download)' - **van der Wardt, Rik(2017)** <https://agilescrumgroup.nl/5-tips-goede-daily-stand-up-meeting/>
- [@vanlier_2018](#): 'Stakeholdermanagement in projecten met Scrum' - **van Lier, Willemijn(2018)** <https://agilescrumgroup.nl/stakeholder-management-matrix-model/>
- [@vanrooden_2015](#): 'Product Backlog Refinement explained (1/3)' - **van Rooden, Stephan(2015)** <https://www.scrum.org/resources/blog/product-backlog-refinement-explained-13>

- [@vanrooden_2015a](#): *'Product Backlog Refinement explained (3/3)'* - **van Rooden, Stephan(2015)** <https://www.scrum.org/resources/blog/product-backlog-refinement-explained-33>
- [@verheyen_2022](#): *'Scrum Glossary'* - **Verheyen, Gunther(2022)** <https://scrumglossary.org/>
- [@visualparadigm_2022](#): *'What are Time-boxed Events in Scrum?'* - **Visual Paradigm, (2022)** <https://www.visual-paradigm.com/scrum/what-are-scrum-time-boxed-events/>
- [@visual-paradigm_2022](#): *'Why Fixed Length Sprints in Scrum?'* - **visual-paradigm, (2022)** <https://www.visual-paradigm.com/scrum/why-fixed-length-of-sprints-in-scrum/>
- [@vrielink_2020](#): *'Hoe werken story points?'* - **Vrielink, Martijn(2020)** <https://www.incentro.com/nl-NL/blog/story-points-hoe-werken-ze-eigenlijk>
- [@zentaoteam_2022a](#): *'What Are The Complete Scrum Artifacts? - Agile - ZenTao'* - **ZenTao team,(2022)** <https://www.zentao.pm/blog/What-are-the-complete-Scrum-artifacts-1102.html>

-
1. By Planbox - Own work, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=19543504> ↩
 2. By PierreSelim - Own work, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=17336884> ↩
 3. Micromanagement is een managementstijl waarbij een leidinggevende zeer nauw toezicht houdt op en vaak te gedetailleerd betrokken is bij het werk van zijn of haar medewerkers. Dit betekent doorgaans dat een manager niet alleen bepaalt wat er gedaan moet worden, maar ook precies hoe en wanneer het moet gebeuren, soms tot op het punt dat elke kleine stap wordt gecontroleerd en voorgeschreven. Dit kan leiden tot een gebrek aan autonomie en creativiteit bij werknemers, en vaak tot frustratie en een lagere werktevredenheid. ↩